

Predicting the functional status of water wells in Tanzania

Business understanding

Water is a basic need and right for all human beings and access to clean and safe water remains a significant challenge in Tanzania, a country with a population exceeding 57 million people. While thousands of water points (wells and pumps) have been installed across the nation, not all of them function reliably. Many are partially functional, in need of repair, or completely non-functional. This undermines efforts to provide consistent access to water, particularly in rural areas. Using data on pump type, installation year, geographic location, and other related attributes, we can build a predictive model that classifies the condition of a water well. Such a system would help prioritize repairs, allocate resources effectively, and guide future infrastructure investments.

Problem statement

The traditional model for water infrastructure management in Tanzania is reactive: a pump is serviced or repaired only after it has completely failed. This approach is inherently inefficient and costly, leading to prolonged periods without access to water for communities and significant expense for emergency repairs.

This project is determined to build predictive model that represents a fundamental paradigm shift to proactive and data driven maintenance. and by leveraging historical data and machine learning, the model can anticipate which water points are at risk of failure or are already in a state of disrepair, enabling maintenance crews to intervene before a complete breakdown occurs.

Objectives

The most critical and actionable objective of this project is to:

- Accurately identify water points that are currently functional but need repair
- Identify patterns in non-functional wells to inform better design, placement, and maintenance of new water points.

Metrics of success

The success of the model will be evaluated using classification metrics:

- Accuracy
- precision and recall
- F1 score
- Confusion matrix
- ROC-AUC

Data understanding

The dataset from the Tanzania Ministry of Water provides records of waterpoints across the country. Each record describes the physical, technical, financial, and management aspects of a waterpoint, alongside its operating condition (target variable: functional, needs repair, or non-functional).

The features provided can be grouped as follows:

1. Water availability & technical
 - amount_tsh: total static head (amount of water available)
 - gps_height: altitude of the well
 - construction_year: year the waterpoint was built
 - num_private: private codes/indicators (often missing or unused)
2. Location & geography
 - longitude, latitude: GPS coordinates
 - basin: geographic water basin
 - subvillage, region, region_code, district_code, lga, ward: administrative/geographic locations
3. Funding & installation
 - funder: who funded the well
 - installer: organization that installed the well
 - scheme_name: water scheme name
 - scheme_management: operator of the scheme
4. Management & governance
 - management, management_group: how the waterpoint is managed
 - public_meeting: whether a public meeting was held
 - permit: whether the waterpoint is permitted



- recorded_by: group recording the data
5. Water extraction & source
 - extraction_type, extraction_type_group, extraction_type_class: type of pump/extraction system
 - source, source_type, source_class: water source information
 6. Water quality & quantity
 - water_quality, quality_group: reported quality of the water
 - quantity, quantity_group: availability of water
 7. Payment & financials
 - payment, payment_type: cost of water use
 8. Waterpoint details
 - waterpoint_type, waterpoint_type_group: kind of waterpoint
 - wpt_name: waterpoint name (if present)
 - date_recorded: when the record was entered
 - population: number of people depending on the waterpoint

Data preparation

```
In [387]: # importing the relevant libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.preprocessing import OneHotEncoder, LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression, ElasticNet, ElasticNetCV
from imblearn.over_sampling import SMOTE
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_auc_score, roc_curve, auc
from sklearn.preprocessing import label_binarize
from sklearn.model_selection import GridSearchCV
from xgboost import XGBClassifier
from sklearn.model_selection import RandomizedSearchCV
```

```
In [388]: # Loading the datasets
trainig_values_df= pd.read_csv("training_set_values.csv", index_col= "id")
training_labels_df= pd.read_csv("training_set_labels.csv", index_col= "id")
test_values_df= pd.read_csv("test_set_values.csv")
```

```
In [389]: # reading the training set values df
trainig_values_df.shape
```

Out[389]: (59400, 39)

```
In [390]: training_labels_df.head()
```

Out[390]:

	status_group
id	
69572	functional
8776	functional
34310	functional
67743	non functional
19728	functional

In [391]:

test_values_df.head()

Out[391]:

	id	amount_tsh	date_recorded	funder	gps_height	installer	longitude	latitude	wpt_name	num_private	...	payment_type	wat
0	50785	0.0	2013-02-04	Dmdd	1996	DMDD	35.290799	-4.059696	Dinamu Secondary School	0	...	never pay	
1	51630	0.0	2013-02-04	Government Of Tanzania	1569	DWE	36.656709	-3.309214	Kimnyak	0	...	never pay	
2	17168	0.0	2013-02-01	NaN	1567	NaN	34.767863	-5.004344	Puma Secondary	0	...	never pay	
3	45559	0.0	2013-01-22	Finn Water	267	FINN WATER	38.058046	-9.418672	Kwa Mzee Pange	0	...	unknown	
4	49871	500.0	2013-03-27	Bruder	1260	BRUDER	35.006123	-10.950412	Kwa Mzee Turuka	0	...	monthly	

5 rows × 40 columns

In [392]:

```
# printing the shapes all together
print("Observation:")
print(f"The training set values has {trainig_values_df.shape[0]} records and {trainig_values_df.shape[1]} features")
print(f"The training set values has {training_labels_df.shape[0]} records and {training_labels_df.shape[1]} features")
print(f"The training set values has {test_values_df.shape[0]} records and {test_values_df.shape[1]} features")
```

Observation:
The training set values has 59400 records and 39 features
The training set values has 59400 records and 1 features
The training set values has 14850 records and 40 features

In [393]:

```
# joining training Labels and training values df
train_df= pd.concat([trainig_values_df, training_labels_df], axis= 1)
```

In [394]:

```
# checking the shape of the joined data
train_df.shape
```

Out[394]: (59400, 40)

In [395]:

```
# checking the df
train_df.head()
```

Out[395]:

	amount_tsh	date_recorded	funder	gps_height	installer	longitude	latitude	wpt_name	num_private	basin	...	water_quality
id												
69572	6000.0	2011-03-14	Roman	1390	Roman	34.938093	-9.856322	none	0	Lake Nyasa	...	soft
8776	0.0	2013-03-06	Grumeti	1399	GRUMETI	34.698766	-2.147466	Zahanati	0	Lake Victoria	...	soft
34310	25.0	2013-02-25	Lottery Club	686	World vision	37.460664	-3.821329	Kwa Mahundi	0	Pangani	...	soft
67743	0.0	2013-01-28	Unicef	263	UNICEF	38.486161	-11.155298	Zahanati Ya Nanyumbu	0	Ruvuma / Southern Coast	...	soft
19728	0.0	2011-07-13	Action In A	0	Artisan	31.130847	-1.825359	Shuleni	0	Lake Victoria	...	soft

5 rows × 40 columns

Cleaning columns with null values

In [396]: `# checking datatypes`
`train_df.info()`

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 59400 entries, 69572 to 26348
Data columns (total 40 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   amount_tsh                           59400 non-null  float64
1   date_recorded                        59400 non-null  object
2   funder                               55765 non-null  object
3   gps_height                           59400 non-null  int64
4   installer                            55745 non-null  object
5   longitude                            59400 non-null  float64
6   latitude                             59400 non-null  float64
7   wpt_name                             59400 non-null  object
8   num_private                          59400 non-null  int64
9   basin                               59400 non-null  object
10  subvillage                           59029 non-null  object
11  region                               59400 non-null  object
12  region_code                          59400 non-null  int64
13  district_code                       59400 non-null  int64
14  lga                                  59400 non-null  object
15  ward                                 59400 non-null  object
16  population                           59400 non-null  int64
17  public_meeting                      56066 non-null  object
18  recorded_by                         59400 non-null  object
19  scheme_management                   55523 non-null  object
20  scheme_name                         31234 non-null  object
21  permit                              56344 non-null  object
22  construction_year                   59400 non-null  int64
23  extraction_type                     59400 non-null  object
24  extraction_type_group                59400 non-null  object
25  extraction_type_class                59400 non-null  object
26  management                           59400 non-null  object
27  management_group                    59400 non-null  object
28  payment                             59400 non-null  object
29  payment_type                        59400 non-null  object
30  water_quality                       59400 non-null  object
31  quality_group                       59400 non-null  object
32  quantity                            59400 non-null  object
33  quantity_group                      59400 non-null  object
34  source                              59400 non-null  object
35  source_type                         59400 non-null  object
36  source_class                        59400 non-null  object
37  waterpoint_type                     59400 non-null  object
38  waterpoint_type_group               59400 non-null  object
39  status_group                        59400 non-null  object
dtypes: float64(3), int64(6), object(31)
memory usage: 18.6+ MB
```

Observation:

There are a total of 9 numerical columns and 31 columns with categorical values

```
In [397]: # checking for missing values in training set values df
missing_values= train_df.isna().sum()

missing_counts= missing_values.sort_values(ascending= False)
missing_counts
```

```
Out[397]: scheme_name      28166
scheme_management    3877
installer            3655
funder              3635
public_meeting      3334
permit              3056
subvillage           371
gps_height           0
longitude            0
latitude             0
wpt_name             0
num_private          0
recorded_by          0
region              0
region_code          0
district_code        0
lga                  0
ward                 0
population           0
date_recorded        0
basin                0
status_group         0
waterpoint_type_group 0
construction_year    0
waterpoint_type      0
source_class         0
source_type          0
source               0
quantity_group       0
quantity             0
quality_group        0
water_quality        0
payment_type         0
payment              0
management_group     0
management           0
extraction_type_class 0
extraction_type_group 0
extraction_type      0
amount_tsh           0
dtype: int64
```

```
In [398]: # checking columns with the missing values
missing_cols = train_df.columns[train_df.isnull().any()].tolist()

print(f"Columns with missing values: {missing_cols}")
```

```
Columns with missing values: ['funder', 'installer', 'subvillage', 'public_meeting', 'scheme_management', 'scheme_name', 'permit']
```

Observation:

The train_df dataset has less columns with missing values, just a few columns like scheme name, scheme management, installer, funder, public meeting, permit and subvillage with some missing values which can be handled.

```
In [399]:  # checking for value counts in columns with missing values  
          for col in missing_cols:  
             print(f"\nColumn: {col}")  
             print(train_df[col].value_counts(dropna=False))
```

Column: funder
 Government Of Tanzania 9084
 NaN 3635
 Danida 3114
 Hesawa 2202
 Rwssp 1374

...
 Noeli Mahobokela 1
 Batist Church 1
 Unice/ Cspd 1
 Tlc/seleman Mang'ombe 1
 Bkhws 1

Name: funder, Length: 1898, dtype: int64

Column: installer
 DWE 17402
 NaN 3655
 Government 1825
 RWE 1206
 Commu 1060

...
 OMARY MONA 1
 Rudri 1
 KKKT Canal 1
 Billy Phillips 1
 Got 1

Name: installer, Length: 2146, dtype: int64

Column: subvillage
 Madukani 508
 Shuleni 506
 Majengo 502
 Kati 373
 NaN 371

...
 Isenga Center 1
 Kasolo 1
 Rangwa 1
 Afrika Yakati 1
 Kumweluro A 1

Name: subvillage, Length: 19288, dtype: int64

Column: public_meeting
 True 51011
 False 5055
 NaN 3334

Name: public_meeting, dtype: int64

Column: scheme_management
 VWC 36793
 WUG 5206
 NaN 3877
 Water authority 3153
 WUA 2883
 Water Board 2748
 Parastatal 1680
 Private operator 1063
 Company 1061
 Other 766
 SWC 97
 Trust 72
 None 1

Name: scheme_management, dtype: int64

Column: scheme_name
 NaN 28166
 K 682
 None 644
 Borehole 546
 Chalinze wate 405

...
 Mradi wa maji wa kipampa 1
 Magoma water suoply 1
 Mradi wa maji wa Kakola 1
 Mwigimbi piped scheme 1
 Center 1

Name: scheme_name, Length: 2697, dtype: int64

Column: permit

```

True      38852
False     17492
NaN        3056
Name: permit, dtype: int64

```

Cleaning installer column

```

In [400]: # filling in missing values in installer column
train_df['installer'] = train_df['installer'].str.lower().str.strip() # normalizing text
train_df["installer"].value_counts()[:20]

```

```

Out[400]: dwe      17405
government  1891
hesawa      1395
rwe         1206
commu       1065
danida      1050
district council  961
kkkt        910
0           777
central government  763
tcrs         707
world vision  699
danid        624
ces          610
community    555
gover        511
amref        443
lga          413
tasaf        411
wedeco       398
Name: installer, dtype: int64

```

```

In [401]: # mapping to unify common misspellings
mapping = {
    'world viission': 'world vision',
    'world division': 'world vision',
    'would bank': 'world bank',
    'word bank': 'world bank',
    'world banks': 'world bank',
    'word divisio': 'world division',
    'danid': 'danida',
    'governm': 'government',
    'cental government': 'central government',
    'central govt': 'central government',
    'govern': 'government',
    'serikali': 'government',
    'oxfam': 'oxfam',
    'unisef': 'unicef',
    'commu': 'community',
    'gove': 'government',
    'finw': 'fini water',
    'fw': 'fini water',
    'finwater': 'fini water',
    'finn water': 'fini water',
    'fin water': 'fini water',
    'gover': 'government',
    '0': 'unknown',
    '-': 'unknown',
    'acra': 'accra',
    'centr': 'central government',
    'communit': 'community',
    'comunity': 'community',
    'cartas': 'caritas',
    'conce': 'concern',
    'not known': 'unknown'}

```

```

In [402]: # replacing in the mapped values
train_df["installer"] = train_df["installer"].replace(mapping)
# test_values_df["installer"] = test_values_df["installer"].replace(mapping)

```

```

In [403]: train_df['installer'].fillna("other", inplace= True)

```



```
In [404]: # Keep top 20 installers and group the rest into "other".
top_installers = train_df["installer"].value_counts().nlargest(20).index
train_df["installer"] = train_df["installer"].apply(lambda x: x if x in top_installers else "other")
train_df["installer"].unique().tolist()
```

```
Out[404]: ['other',
'world vision',
'dwe',
'danida',
'wedeco',
'district council',
'unknown',
'central government',
'community',
'lga',
'government',
'kkkt',
'rwe',
'hesawa',
'ces',
'tasaf',
'fini water',
'amref',
'dmdd',
'tcrs']
```

cleaning funder column

```
In [405]: # checking value counts of the top 20
train_df["funder"].value_counts()[:20]
```

```
Out[405]: Government Of Tanzania    9084
Danida                             3114
Hesawa                             2202
Rwssp                              1374
World Bank                         1349
Kkkt                               1287
World Vision                       1246
Unicef                             1057
Tasaf                              877
District Council                   843
Dhv                                829
Private Individual                 826
Dwsp                               811
0                                  777
Norad                             765
Germany Republi                   610
Tcrs                              602
Ministry Of Water                 590
Water                             583
Dwe                               484
Name: funder, dtype: int64
```

```
In [406]: # normalizing values in funder column for both training

train_df['funder'] = train_df['funder'].str.lower().str.strip() # normalizing text
# test_values_df['funder'] = test_values_df['funder'].str.lower().str.strip()
```

```
In [407]: # Listing the unique value to identify misspelled mistakes
train_df["funder"].unique().tolist()
```

```
Out[407]: ['roman',
'grumeti',
'lottery club',
'unicef',
'action in a',
'mkinga distric coun',
'dwsp',
'rwssp',
'wateraid',
'isingiro ho',
'private',
'danida',
'world vision',
'lawatefuka water supply',
'biore',
'rudep',
'hesawa',
'twe',
'isf',
'...
```

```
In [ ]:
```

```
In [408]: # mapping to unify common misspellings
mapping = {
    "oxfarm": "oxfam",
    "oxfam gb": "oxfam",
    "govt of tanzania": "government of tanzania",
    "tanzania government": "government of tanzania",
    "germany republi": "german republic",
    "gvt": "government of tanzania",
    "serikaru": "government of tanzania",
    "serikali": "government of tanzania",
    "serikari": "government of tanzania",
    "usa embassy": "us embassy",
    "ubalozi wa marekani": "us embassy",
    "u.s.a": "usa",
    "world vision": "world vision",
    "vision world": "world vision",
    "roman": "roman catholic",
    "rc church": "roman catholic",
    "catholic church": "roman catholic",
    "rc ch": "roman catholic",
    "roman cathoric same": "roman catholic",
    "roman church": "roman catholic",
    "roman cathoric": "roman catholic",
    "kanisa katoliki": "roman catholic",
    "roman cathoric church": "roman catholic",
    "roman ca": "roman catholic",
    "nethe": "netherlands",
    "nethalan": "netherlands",
    "finw": "fini water",
    "finn water": "fini water",
    "finwater": "fini water",
    "fin water": "fini water",
    "commu": "community",
    "unicef": "unicef",
    "hamref": "amref",
    "village govt": "village government",
    "serkali ya kijiji": "village government",
    "priva": "private individual",
    "private": "private individual",
    "0": "unknown",
    "not known": "unknown"}
```

```
In [409]: train_df["funder"] = train_df["funder"].replace(mapping)
# test_values_df['funder'] = test_values_df["funder"].replace(mapping)
```

```
In [410]: # filling null values unknown
train_df["funder"].fillna("other", inplace= True)
```

```
In [411]: # Keep top 20 funders and group the rest into "other".
top_funders = train_df["funder"].value_counts().nlargest(20).index
train_df["funder"] = train_df["funder"].apply(lambda x: x if x in top_funders else "other")
train_df["funder"].unique().tolist()
```

```
Out[411]: ['roman catholic',
'other',
'unicef',
'dwsp',
'rwssp',
'private individual',
'danida',
'world vision',
'hesawa',
'government of tanzania',
'unknown',
'district council',
'kkkt',
'norad',
'german republic',
'netherlands',
'tasaf',
'world bank',
'fini water',
'dhv']
```

```
In [412]: # top_funders = test_values_df["funder"].value_counts().nlargest(20).index
# test_values_df['funder'] = test_values_df['funder'].apply(lambda x: x if x in top_funders else 'other')
# test_values_df["funder"].unique().tolist()
```

```
In [413]: # checking null values in installer column
train_df.isna().sum()
```

```
Out[413]: amount_tsh          0
date_recorded              0
funder                    0
gps_height                 0
installer                 0
longitude                 0
latitude                 0
wpt_name                  0
num_private               0
basin                     0
subvillage                371
region                    0
region_code               0
district_code             0
lga                       0
ward                      0
population                0
public_meeting            3334
recorded_by               0
scheme_management         3877
scheme_name               28166
permit                    3056
construction_year         0
extraction_type           0
extraction_type_group     0
extraction_type_class     0
management               0
management_group         0
payment                   0
payment_type              0
water_quality             0
quality_group             0
quantity                  0
quantity_group            0
source                    0
source_type               0
source_class              0
waterpoint_type           0
waterpoint_type_group     0
status_group              0
dtype: int64
```

Cleaning public meeting column

```
In [414]: # checking column values
train_df.public_meeting.value_counts(dropna=False)
```

```
Out[414]: True      51011
False     5055
NaN       3334
Name: public_meeting, dtype: int64
```

```
In [415]: # filling NaN with unknown
train_df.public_meeting.fillna('unknown', inplace=True)
```

```
In [416]: # Changing all the values to be strings instead of boolean
train_df.public_meeting = train_df.public_meeting.astype('str')
```

Cleaning permit column

```
In [417]: # checking column values
train_df.permit.value_counts(dropna=False)
```

```
Out[417]: True      38852
False     17492
NaN       3056
Name: permit, dtype: int64
```

```
In [418]: # replacing NaN with unknown
train_df.permit.fillna('unknown', inplace=True)
```

```
In [419]: # Changing all the values to be strings instead of boolean
train_df.permit = train_df.permit.astype('str')
```

Cleaning scheme name and scheme management columns

```
In [420]: train_df.scheme_name.value_counts()
```

```
Out[420]: K      682
None      644
Borehole   546
Chalinze wate  405
M          400
...
Usambara Mountain      1
Lunguo water supply     1
Community               1
RC Water Supply         1
LANCH                   1
Name: scheme_name, Length: 2696, dtype: int64
```

```
In [421]: # dropping scheme name column because the information is not valuable and also it has alot of missing values
train_df.drop('scheme_name', axis=1, inplace=True)
```

```
In [422]: # checking scheme management counts
train_df.scheme_management.value_counts(dropna=False)
```

```
Out[422]: VWC          36793
WUG          5206
NaN          3877
Water authority  3153
WUA          2883
Water Board    2748
Parastatal    1680
Private operator 1063
Company        1061
Other          766
SWC            97
Trust          72
None           1
Name: scheme_management, dtype: int64
```

```
In [423]: # filling in missing values with other
train_df.scheme_management.fillna('None', inplace=True)
```

```
In [424]: # checking for effectiveness
train_df["scheme_management"].value_counts()
```

```
Out[424]: VWC          36793
WUG          5206
None          3878
Water authority  3153
WUA          2883
Water Board    2748
Parastatal    1680
Private operator 1063
Company        1061
Other          766
SWC            97
Trust          72
Name: scheme_management, dtype: int64
```

```
In [425]: # checking subvillage column
train_df["subvillage"].value_counts()
```

```
Out[425]: Madukani      508
Shuleni      506
Majengo      502
Kati         373
Mtakuja      262
...
Isenga Center  1
Kasolo         1
Rangwa         1
Afrika Yakati  1
Mangomboli     1
Name: subvillage, Length: 19287, dtype: int64
```

```
In [426]: # dropping the column because it has missing values and also wont be necessary in the modeling
train_df.drop("subvillage", axis=1, inplace=True)
```

Counterchecking columns without null values

```
In [427]: # checking all columns value counts
for col in train_df.columns:
    print(f"\nColumn: {col}")
    print(train_df[col].value_counts(dropna=False).head(10))
```

```
Column: amount_tsh
0.0      41639
500.0     3102
50.0      2472
1000.0    1488
20.0      1463
200.0     1220
100.0      816
10.0       806
30.0       743
2000.0     704
Name: amount_tsh, dtype: int64
```

```
Column: date_recorded
2011-03-15    572
2011-03-17    558
2013-02-03    546
2011-03-14    520
2011-03-16    512
```

amount column

```
In [428]: # changing amount column to interger
train_df.amount_tsh = train_df.amount_tsh.astype('int64')
```

date recorded

```
In [429]: # converting date_recorded to date time format
train_df['date_recorded'] = pd.to_datetime(train_df['date_recorded'])
train_df["date_recorded"].dt.year.value_counts(dropna=False)
```

```
Out[429]: 2011    28674
          2013    24271
          2012     6424
          2004        30
          2002         1
          Name: date_recorded, dtype: int64
```

Observation:

The above information shows that the number of wells recorded increases over time. with the highest number of wells recorded (about 28674) in 2011 and 2013, while the lowest number was recorded in 2004 (30) and 2002 (1) which suggests that the well construction efforts begun much later. is could possibly be because of the emergence funding from the NGO

```
In [430]: # dropping date recorded column because it is not valuable
train_df.drop('date_recorded', axis=1, inplace=True)
```

Gps_height

```
In [431]: # rechecking gps_gps_heigh value_counts
train_df["gps_height"].value_counts(dropna=False)
```

```
Out[431]: 0      20438
          -15       60
          -16       55
          -13       55
          -20       52
          ...
          2285        1
          2424        1
          2552        1
          2413        1
          2385        1
          Name: gps_height, Length: 2428, dtype: int64
```

Observation:

This values show the range of altitudes where the water points are located from negative to altitude above 2000m Most number of water points are located in the areas of 0 altitude that is near or at the sea level.

```
In [432]: # counter checking value counts
train_df["longitude"].value_counts(dropna=False)
```

```
Out[432]: 0.000000    1812
          37.540901      2
          33.010510      2
          39.093484      2
          32.972719      2
          ...
          37.579803      1
          33.196490      1
          34.017119      1
          33.788326      1
          30.163579      1
          Name: longitude, Length: 57516, dtype: int64
```

Observation: The longitude of 0 is nearly an impossible longitude in area like tanzania. These 0 values could be the placeholder for missing longitude values we need to find a way of imputing the 0 values. the one option is to replace them with null values and later impute with the median of the region after rgrouping by region

```
In [433]: train_df["latitude"].value_counts()
```

```
Out[433]: -2.000000e-08    1812
          -6.985842e+00      2
          -3.797579e+00      2
          -6.981884e+00      2
          -7.104625e+00      2
          ...
          -5.726001e+00      1
          -9.646831e+00      1
          -8.124530e+00      1
          -2.535985e+00      1
          -2.598965e+00      1
          Name: latitude, Length: 57517, dtype: int64
```

Observation:

There are a lot of zero values for latitude and longitude which are not possible for Tanzania's location.

```
In [434]: # number of wells constructed per year
train_df["construction_year"].value_counts(dropna=False)
```

```
1994    738
1972    708
1974    676
1997    644
1992    640
1993    608
2001    540
1988    521
1983    488
1975    437
1986    434
1976    414
1970    411
1991    324
1989    316
1987    302
1981    238
1977    202
1979    192
1973    184
```

Observation:

The 0 construction year is an error during data entry and therefore it has the highest values(20709), this because there is no 0 years it should a value greater the zero to indicate the year the well was constructed.

```
In [435]: # handling zeros in the construction year column
          """ Construction year column has alot of 0 values as years of construction shown in the value_counts above."""

          # finding exact number of zero years
          zero_years= (train_df["construction_year"]== 0).sum()
          print(f"Number of records with construction_year = 0: {zero_years}")
```

Number of records with construction_year = 0: 20709

```
In [436]: # replacing 0 in construction years with null values
          train_df['construction_year'] = train_df['construction_year'].replace(0, np.nan)

          # # calculating well age using date_recorded
          # train_df["well_age"] = train_df['date_recorded'].dt.year - train_df["construction_year"]

          # train_df['well_age'].value_counts()

          # calculating the median of the construction year where years is not zero
          year_median = int(train_df[train_df['construction_year'] != 0]['construction_year'].median())

          # Filling NaN with the median
          train_df['construction_year'].fillna(year_median, inplace= True)
```



```
In [437]: # checking the column
train_df["construction_year"].value_counts(dropna=False)
```

```
Out[437]: 2000.0    22800
          2010.0     2645
          2008.0     2613
          2009.0     2533
          2007.0     1587
          2006.0     1471
          2003.0     1286
          2011.0     1256
          2004.0     1123
          2012.0     1084
          2002.0     1075
          1978.0     1037
          1995.0     1014
          2005.0     1011
          1999.0      979
          1998.0      966
          1990.0      954
          1985.0      945
          1996.0      811
          1980.0      811
          1984.0      779
          1982.0      744
          1994.0      738
          1972.0      708
          1974.0      676
          1997.0      644
          1992.0      640
          1993.0      608
          2001.0      540
          1988.0      521
          1983.0      488
          1975.0      437
          1986.0      434
          1976.0      414
          1970.0      411
          1991.0      324
          1989.0      316
          1987.0      302
          1981.0      238
          1977.0      202
          1979.0      192
          1973.0      184
          2013.0      176
          1971.0      145
          1960.0      102
          1967.0       88
          1963.0       85
          1968.0       77
          1969.0       59
          1964.0       40
          1962.0       30
          1961.0       21
          1965.0       19
          1966.0       17
Name: construction_year, dtype: int64
```

```
In [438]: # # filling in missing well age values with median
# train_df['well_age'] = train_df['well_age'].fillna(train_df['well_age'].median())
```

```
In [439]: # # checking missing values in well age column
# train_df['well_age'].isna().sum()
```

Wpt_name column

```
In [440]: # checking value_counts
train_df["wpt_name"].value_counts(dropna= False)
```

```
Out[440]: none          3563
Shuleni             1748
Zahanati            830
Msikitini           535
Kanisani            323
...
S/M Kinyag'Erere    1
Kwa Kihemba         1
Kwa Oscar Mathiasi  1
Ilungu C            1
Kwa Richard Kamagare 1
Name: wpt_name, Length: 37400, dtype: int64
```

```
In [441]: # dropping this column as it only contains names of the wells which is not necessary in our analysis
train_df.drop('wpt_name', axis=1, inplace=True)
```

Num_private column

```
In [442]: # crosschecking counts
train_df['num_private'].value_counts(normalize= True)
```

```
Out[442]: 0      0.987256
6      0.001364
1      0.001229
5      0.000774
8      0.000774
...
180    0.000017
213    0.000017
23     0.000017
55     0.000017
94     0.000017
Name: num_private, Length: 65, dtype: float64
```

```
In [443]: # dropping num column since we would not need in our analysis
train_df.drop("num_private", axis= 1, inplace= True)
```

basin column

```
In [444]: # checking for value_counts
train_df["basin"].value_counts(dropna= False)
```

```
Out[444]: Lake Victoria    10248
Pangani                   8940
Rufiji                    7976
Internal                  7785
Lake Tanganyika           6432
Wami / Ruvu               5987
Lake Nyasa                5085
Ruvuma / Southern Coast   4493
Lake Rukwa                2454
Name: basin, dtype: int64
```

Region and region code columns

```
In [445]: # checking values counts
train_df[["region", "region_code"]].value_counts()
```

```
Out[445]: region      region_code
Iringa      11           5294
Shinyanga   17           4956
Mbeya       12           4639
Kilimanjaro 3           4379
Morogoro     5           4006
Kagera      18           3316
Mwanza      19           3047
Arusha       2           3024
Kigoma       16           2816
Ruvuma       10           2640
Tanga        4           2513
Dodoma       1           2201
Singida      13           2093
Mara         20           1969
Tabora       14           1959
Rukwa        15           1808
Pwani         6           1609
Manyara      21           1583
Lindi        80           1238
Pwani        60           1025
Mtwara       90           917
Dar es Salaam 7           805
Mtwara       99           423
              9           390
Arusha       24           326
Lindi         8           300
Mwanza       17           55
Tanga         5           34
Shinyanga    14           20
Lindi        18            8
Shinyanga    11            6
Pwani        40            1
dtype: int64
```

Observation:

Since we have a manageable amount of regions, we would have to drop the region code column because we can identify the region by its name

```
In [446]: # dropping region_code
train_df.drop('region_code', axis=1, inplace=True)
```

District code

```
In [447]: # checking value counts
train_df["district_code"].value_counts()
```

```
Out[447]: 1      12203
          2      11173
          3       9998
          4       8999
          5       4356
          6       4074
          7       3343
          8       1043
         30        995
         33        874
         53        745
         43        505
         13        391
         23        293
         63        195
         62        109
         60         63
          0         23
         80         12
         67          6
Name: district_code, dtype: int64
```

```
In [448]: # dropping the column as it will not be used elsewhere
train_df.drop('district_code', axis=1, inplace=True)
```

Lga Column

```
In [449]: # checking column values
train_df['lga'].value_counts(dropna = False, normalize=True)
```

```
Out[449]: Njombe      0.042138
Arusha Rural  0.021077
Moshi Rural   0.021061
Bariadi       0.019815
Rungwe        0.018620
...
Moshi Urban   0.001330
Kigoma Urban  0.001195
Arusha Urban   0.001061
Lindi Urban   0.000354
Nyamagana     0.000017
Name: lga, Length: 125, dtype: float64
```

Ward

```
In [450]: # checking column values
train_df["ward"].value_counts()
```

```
Out[450]: Igosi      307
Imalinyi    252
Siha Kati   232
Mdandu      231
Nduruma     217
...
Themi       1
Thawi       1
Uchindile   1
Korongoni   1
Mitole      1
Name: ward, Length: 2092, dtype: int64
```

```
In [451]: # dropping the ward column
train_df.drop('ward', axis=1, inplace=True)
```

Population

```
In [452]: # value counts
train_df.population.value_counts(dropna=False)
```

```
Out[452]: 0      21381
1       7025
200     1940
150     1892
250     1681
...
3241     1
1960     1
1685     1
2248     1
1439     1
Name: population, Length: 1049, dtype: int64
```

Observation:

will keep the zero values for population because there's no evidence that a well cannot exist in an area with no registered population.

Recorded_by column

```
In [453]: # Looking at column values
train_df.recorded_by.value_counts(dropna=False)
```

```
Out[453]: GeoData Consultants Ltd    59400
Name: recorded_by, dtype: int64
```

Observation:

As per the value counts there seems to be only one firm that did data recording. so for that reason we would not need this column

```
In [454]: # dropping the column
train_df.drop("recorded_by", axis=1, inplace=True)
```

Extraction columns

```
In [455]: # checking the extraction type, extraction type group and extraction type class
# extraction_type
train_df["extraction_type"].value_counts()
```

```
Out[455]: gravity                26780
nira/tanira                    8154
other                          6430
submersible                    4764
swn 80                         3670
mono                          2865
india mark ii                  2400
afridev                       1770
ksb                            1415
other - rope pump              451
other - swan 81                229
windmill                      117
india mark iii                 98
cemo                           90
other - play pump              85
walimi                         48
climax                        32
other - mkulima/shinyanga      2
Name: extraction_type, dtype: int64
```

```
In [456]: # extraction type_group
train_df["extraction_type_group"].value_counts()
```

```
Out[456]: gravity                26780
nira/tanira                    8154
other                          6430
submersible                    6179
swn 80                         3670
mono                          2865
india mark ii                  2400
afridev                       1770
rope pump                      451
other handpump                 364
other motorpump               122
wind-powered                  117
india mark iii                 98
Name: extraction_type_group, dtype: int64
```

```
In [457]: # extraction type class
train_df["extraction_type_class"].value_counts()
```

```
Out[457]: gravity                26780
handpump                     16456
other                          6430
submersible                    6179
motorpump                     2987
rope pump                      451
wind-powered                  117
Name: extraction_type_class, dtype: int64
```

```
In [458]: # Dropping the columns that contain nearly duplicate information
train_df.drop(columns=['extraction_type_group', 'extraction_type_class'],
               inplace=True)
```

Management and management group columns

```
In [459]: # checking the column values
train_df["management"].value_counts()
```

```
Out[459]: vwc          40507
wug          6515
water board  2933
wua          2535
private operator 1971
parastatal   1768
water authority 904
other        844
company      685
unknown      561
other - school 99
trust        78
Name: management, dtype: int64
```

```
In [460]: # management group column
train_df["management_group"].value_counts()
```

```
Out[460]: user-group  52490
commercial  3638
parastatal   1768
other        943
unknown      561
Name: management_group, dtype: int64
```

Observation:

Since both columns play similar role of management, we will have to drop one column particularly management_group because it provides general management class of the organization as opposed to management column that is a bit specific to the managing organization

```
In [461]: # dropping the management group column
train_df.drop("management_group", axis=1, inplace=True)
```

Payment and payment type columns

```
In [462]: # checking payment column values
train_df["payment"].value_counts()
```

```
Out[462]: never pay          25348
pay per bucket          8985
pay monthly            8300
unknown                8157
pay when scheme fails  3914
pay annually           3642
other                 1054
Name: payment, dtype: int64
```

```
In [463]: # checking payment type column values
train_df["payment_type"].value_counts()
```

```
Out[463]: never pay          25348
per bucket          8985
monthly            8300
unknown            8157
on failure          3914
annually           3642
other              1054
Name: payment_type, dtype: int64
```

Observation:

Both the payment and the payment type consist of same information, we will have to drop one

```
In [464]: # dropping payment type column
train_df.drop("payment", axis= 1, inplace= True)
```

Water quality

```
In [465]: # checking column values
train_df["water_quality"].value_counts()
```

```
Out[465]: soft          50818
salty          4856
unknown        1876
milky           804
coloured        490
salty abandoned  339
fluoride        200
fluoride abandoned  17
Name: water_quality, dtype: int64
```

Quality group

```
In [466]: # check the quality group values
train_df["quality_group"].value_counts()
```

```
Out[466]: good          50818
salty          5195
unknown        1876
milky           804
colored         490
fluoride        217
Name: quality_group, dtype: int64
```

Observation:

The above columns nearly consist of similar information, therefore we will have to drop one

```
In [467]: # dropping quality group
train_df.drop("quality_group", axis= 1, inplace= True)
```

Quantity and quantity group columns

```
In [468]: train_df["quantity"].value_counts()
```

```
Out[468]: enough          33186
insufficient  15129
dry           6246
seasonal      4050
unknown        789
Name: quantity, dtype: int64
```

```
In [469]: # quantity_group column
train_df["quantity_group"].value_counts()
```

```
Out[469]: enough          33186
insufficient  15129
dry           6246
seasonal      4050
unknown        789
Name: quantity_group, dtype: int64
```

```
In [470]: # dropping quantity column as it has simillar information
train_df.drop("quantity_group", axis= 1, inplace= True)
```

Source , source_type, source_class Columns

```
In [471]: train_df[["source", "source_type", "source_class",]].value_counts()
```

```
Out[471]: source          source_type      source_class      count
spring          spring          groundwater      17021
shallow well    shallow well    groundwater      16824
machine dbh     borehole        groundwater      11075
river           river/lake       surface          9612
rainwater harvesting rainwater harvesting surface          2295
hand dtw        borehole        groundwater       874
lake            river/lake       surface           765
dam             dam             surface           656
other           other           unknown           212
unknown         other           unknown            66
dtype: int64
```

Observation:

The columns have pretty similar information, we will drop source type and source class column

```
In [472]: # dropping source_type and source_class columns
train_df.drop(["source_class", "source_type"], axis= 1, inplace= True)
```

Waterpoint_type and waterpoint_group columns

```
In [473]: # checking values
train_df["waterpoint_type"].value_counts()
```

```
Out[473]: communal standpipe      28522
hand pump          17488
other              6380
communal standpipe multiple    6103
improved spring      784
cattle trough        116
dam                   7
Name: waterpoint_type, dtype: int64
```

```
In [474]: # waterpoint_type_group
train_df["waterpoint_type_group"].value_counts()
```

```
Out[474]: communal standpipe      34625
hand pump          17488
other              6380
improved spring      784
cattle trough        116
dam                   7
Name: waterpoint_type_group, dtype: int64
```

```
In [475]: # dropping one columns because they have simillar information
train_df.drop("waterpoint_type_group", axis= 1, inplace= True)
```

```
In [476]: train_df["longitude"].value_counts()
```

```
Out[476]: 0.000000      1812
37.540901         2
33.010510         2
39.093484         2
32.972719         2
...
37.579803         1
33.196490         1
34.017119         1
33.788326         1
30.163579         1
Name: longitude, Length: 57516, dtype: int64
```



```
In [477]: # describing the latitude column
train_df["latitude"].describe()
```

```
Out[477]: count    5.940000e+04
mean      -5.706033e+00
std        2.946019e+00
min       -1.164944e+01
25%       -8.540621e+00
50%       -5.021597e+00
75%       -3.326156e+00
max        -2.000000e-08
Name: latitude, dtype: float64
```

Observation:

All the parameters are correct except for max that shows the well with latitude near zero, which is nearly impossible.

```
In [478]: # Replacing near-zero with NaN
train_df.loc[train_df["latitude"].abs() < 0.1, "latitude"] = np.nan

# Computing region mean latitudes (ignoring NaNs)
region_lat_means = train_df.groupby("region")["latitude"].transform("mean")

# Filling missing values with region mean
train_df["latitude"] = train_df["latitude"].fillna(region_lat_means)
```

```
In [479]: train_df["latitude"].describe()
```

```
Out[479]: count    59400.000000
mean      -5.800779
std        2.808707
min       -11.649440
25%       -8.540621
50%       -5.021597
75%       -3.357580
max        -0.998464
Name: latitude, dtype: float64
```

```
In [480]: train_df["latitude"].value_counts()
```

```
Out[480]: -3.495696    1005
-2.620502     807
-6.992611         2
-7.095131         2
-6.990147         2
...
-2.535985         1
-4.828176         1
-4.624770         1
-8.799655         1
-2.598965         1
Name: latitude, Length: 57518, dtype: int64
```

```
In [ ]: # plotting latitude distribution after cleaning
plt.figure(figsize=(8,6))
train_df["latitude"].hist(bins=50)
plt.title("Latitude Distribution - After Cleaning & Imputation")
plt.xlabel("Latitude")
plt.ylabel("Count")
plt.show()
```

```
In [482]: # Longitude column
train_df["longitude"].describe()
```

```
Out[482]: count    59400.000000
mean      34.077427
std        6.567432
min         0.000000
25%       33.090347
50%       34.908743
75%       37.178387
max       40.345193
Name: longitude, dtype: float64
```

Observation: Just like the latitude column the 0 values are irrelevant.

```
In [483]: # changing latitude and longitude columns with 0 values to NaN
train_df["longitude"] = train_df["longitude"].replace(0, np.nan)
```

```
In [484]: train_df["longitude"].value_counts(dropna=False)
```

```
Out[484]: NaN      1812
37.540901      2
33.010510      2
39.093484      2
32.972719      2
...
37.579803      1
33.196490      1
34.017119      1
33.788326      1
30.163579      1
Name: longitude, Length: 57516, dtype: int64
```

```
In [485]: # Computing region mean longitudes (ignoring NaNs)
region_lon_means = train_df.groupby("region")["longitude"].transform("mean")

# Filling missing values with region mean
train_df["longitude"] = train_df["longitude"].fillna(region_lon_means)
```

```
In [486]: train_df["longitude"].value_counts(dropna=False)
```

```
Out[486]: 33.240121      1005
33.091564      807
38.180538      2
37.540901      2
37.534327      2
...
37.579803      1
33.196490      1
34.017119      1
33.788326      1
35.005922      1
Name: longitude, Length: 57517, dtype: int64
```

```
In [487]: # checking the remaining columns
train_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 59400 entries, 69572 to 26348
Data columns (total 22 columns):
#   Column                Non-Null Count  Dtype
---  -
0   amount_tsh            59400 non-null  int64
1   funder                59400 non-null  object
2   gps_height            59400 non-null  int64
3   installer             59400 non-null  object
4   longitude             59400 non-null  float64
5   latitude             59400 non-null  float64
6   basin                59400 non-null  object
7   region               59400 non-null  object
8   lga                  59400 non-null  object
9   population           59400 non-null  int64
10  public_meeting        59400 non-null  object
11  scheme_management     59400 non-null  object
12  permit               59400 non-null  object
13  construction_year     59400 non-null  float64
14  extraction_type       59400 non-null  object
15  management            59400 non-null  object
16  payment_type          59400 non-null  object
17  water_quality         59400 non-null  object
18  quantity              59400 non-null  object
19  source               59400 non-null  object
20  waterpoint_type       59400 non-null  object
21  status_group          59400 non-null  object
dtypes: float64(3), int64(3), object(16)
memory usage: 10.4+ MB
```

```
In [488]: # checking for null values
train_df.isna().sum()
```

```
Out[488]: amount_tsh      0
funder      0
gps_height  0
installer   0
longitude   0
latitude    0
basin       0
region      0
lga         0
population  0
public_meeting  0
scheme_management  0
permit      0
construction_year  0
extraction_type  0
management  0
payment_type  0
water_quality  0
quantity    0
source      0
waterpoint_type  0
status_group  0
dtype: int64
```

```
In [489]: # checking value counts of the target variable
train_df.status_group.value_counts(dropna=False, normalize=True)
```

```
Out[489]: functional      0.543081
non functional    0.384242
functional needs repair  0.072677
Name: status_group, dtype: float64
```

Observation:

Majority of wells are functional(54%), but there is still a significant proportion that are neither non-functional nor need repair(about 45%) There is a major class imbalance as functional wells are much more common than those that are non-function and also those that need repair We may need SMOTE technique to solve the imbalance problem

EDA

univariate analysis

- The focus is on both numerical features and categorical columns
- we are going to plot histograms, KDE plots and box plots to check distribution and outliers

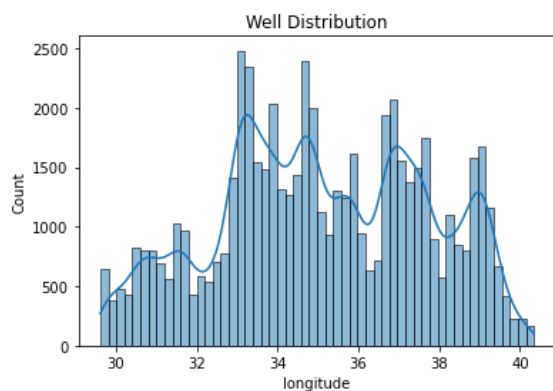
```
In [490]: # identifying numerical columns

numeric_cols = train_df.select_dtypes(include=['int64', 'float64']).columns
numeric_cols
```

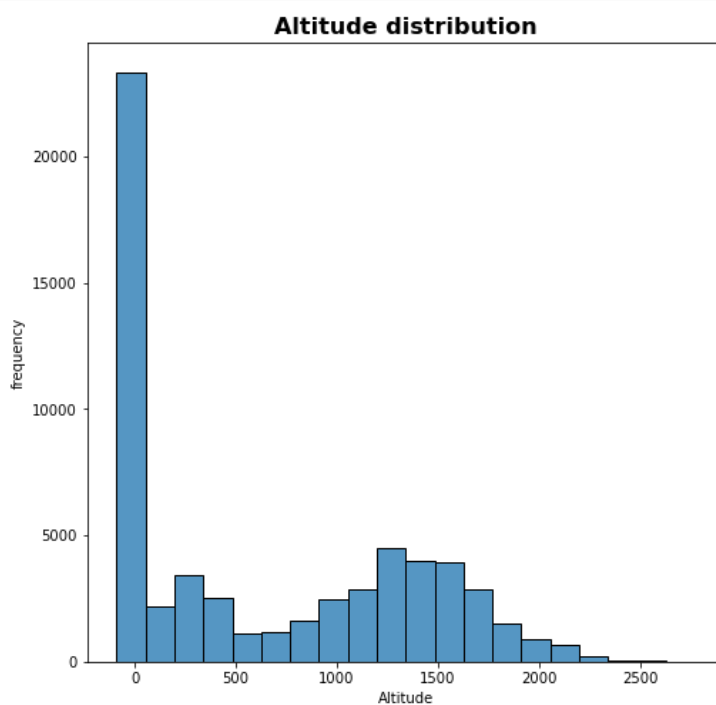
```
Out[490]: Index(['amount_tsh', 'gps_height', 'longitude', 'latitude', 'population',
               'construction_year'],
              dtype='object')
```

```
In [491]: plt.figure(figsize=(6,4))
sns.histplot(train_df['longitude'], kde=True)
plt.title(f"Well Distribution")
```

Out[491]: Text(0.5, 1.0, 'Well Distribution')

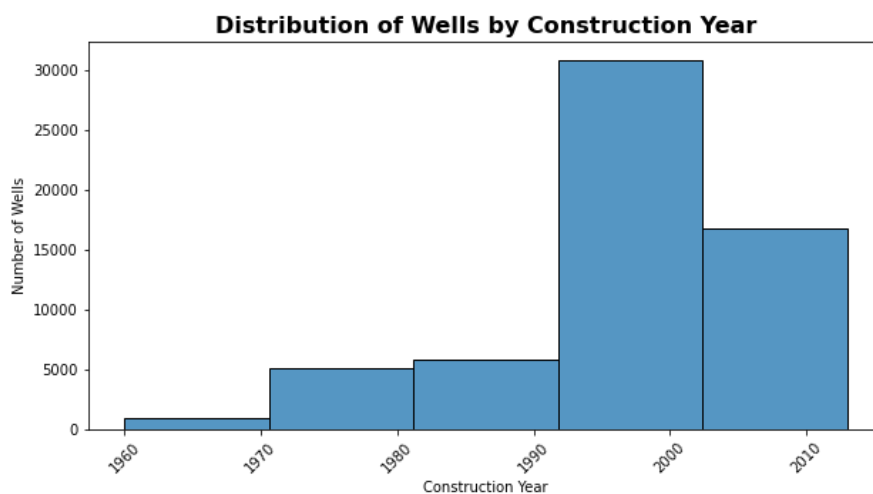


```
In [492]: # gps_height distribution
plt.figure(figsize=(8,8))
sns.histplot(x= "gps_height", data = train_df, bins = 20)
plt.ylabel("frequency")
plt.xlabel("Altitude")
plt.title("Altitude distribution", fontsize=16, fontweight='bold');
```



In [493]: `# distribution of wells constucted per year`

```
plt.figure(figsize=(10,5))
sns.histplot(train_df["construction_year"], bins=5, kde=False)
plt.title("Distribution of Wells by Construction Year", fontsize=16, fontweight='bold')
plt.xlabel("Construction Year")
plt.ylabel("Number of Wells")
plt.xticks(rotation=45)
plt.show()
```

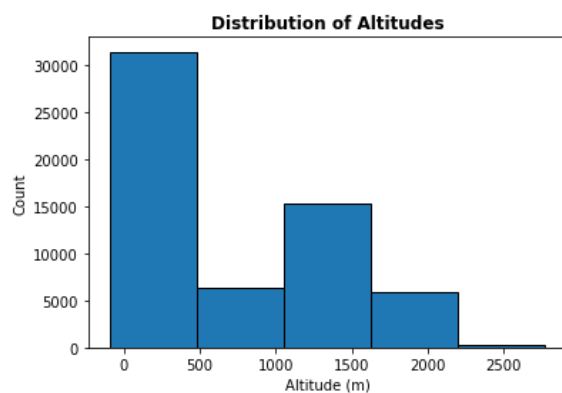


Observation:

The distribution of wells by construction year shows that the number increases over time. the highest number of wells were constructed in the most recent years.

In [494]: `# plotting gps height distribution`

```
plt.hist(train_df["gps_height"], bins=5, edgecolor="black")
plt.xlabel("Altitude (m)")
plt.ylabel("Count")
plt.title("Distribution of Altitudes", fontsize=12, fontweight='bold')
plt.show()
```

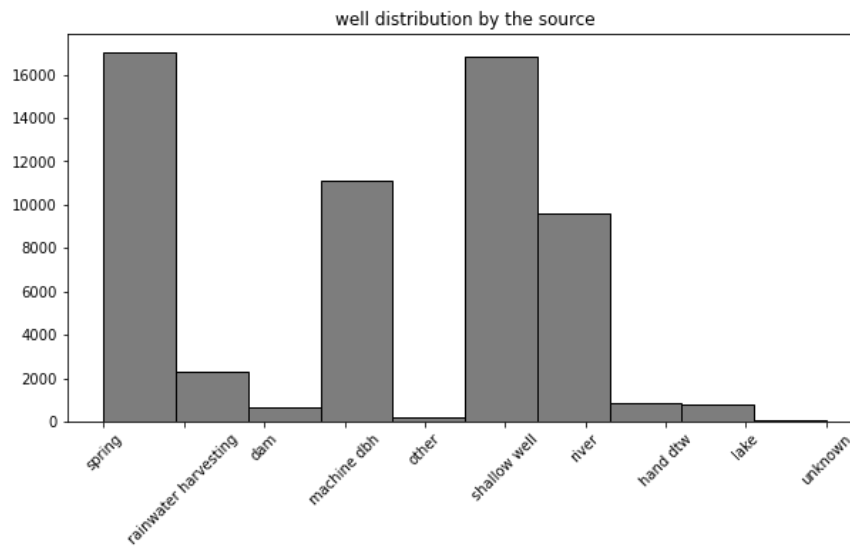


In [495]: `# plotting distribution for categorical`

```
cat_cols= train_df.select_dtypes(include="object").columns
cat_cols
```

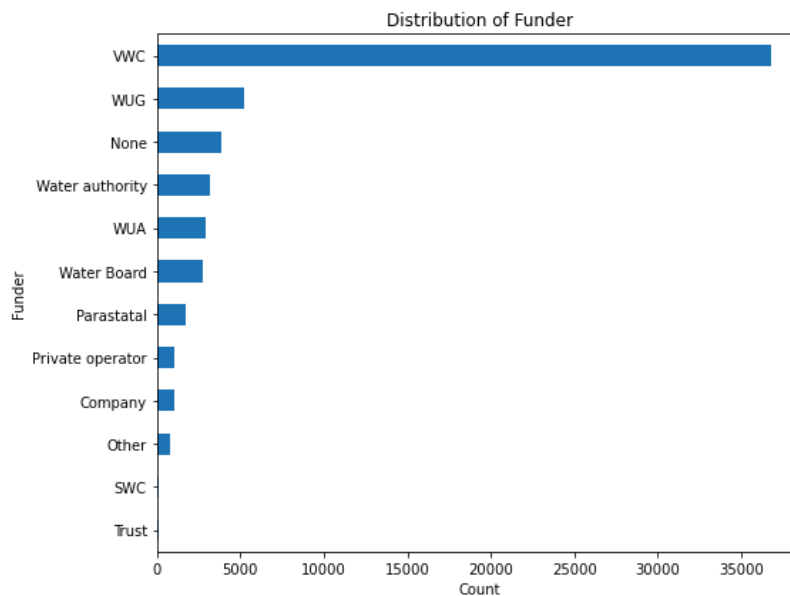
Out[495]: Index(['funder', 'installer', 'basin', 'region', 'lga', 'public_meeting', 'scheme_management', 'permit', 'extraction_type', 'management', 'payment_type', 'water_quality', 'quantity', 'source', 'waterpoint_type', 'status_group'], dtype='object')

```
In [496]: # showing source distribution
plt.figure(figsize=(10,5))
plt.hist(train_df["source"], bins= 10, color= "grey", edgecolor= "black")
plt.xticks(rotation= 45)
plt.title("well distribution by the source");
```



```
In [497]: # distribution of scheme management
counts = train_df['scheme_management'].value_counts().sort_values(ascending= True)

plt.figure(figsize=(8, 6))
counts.plot(kind='barh') # horizontal bar chart
plt.title("Distribution of Funder")
plt.xlabel("Count")
plt.ylabel("Funder")
plt.tight_layout()
plt.show()
```

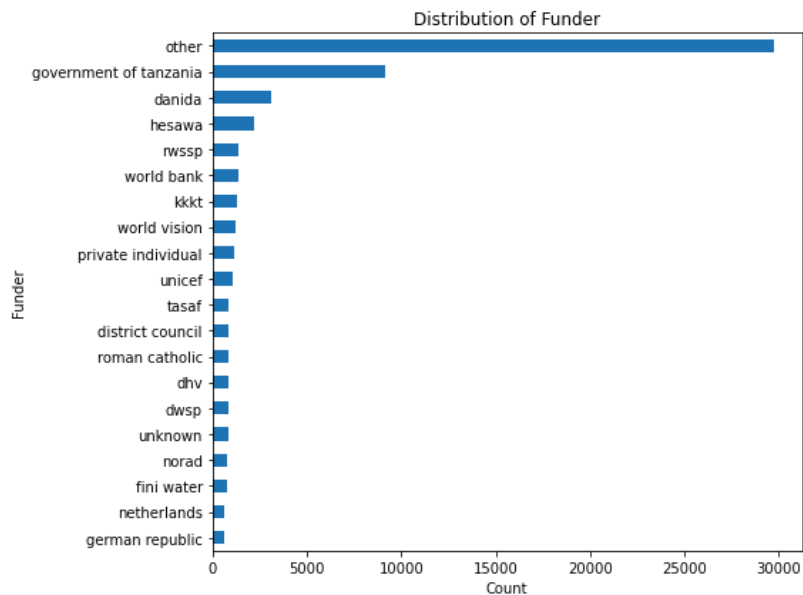


Observation:

WWC are the largest operators of waterpoint operating about (more than 35000) wells across Tanzania.

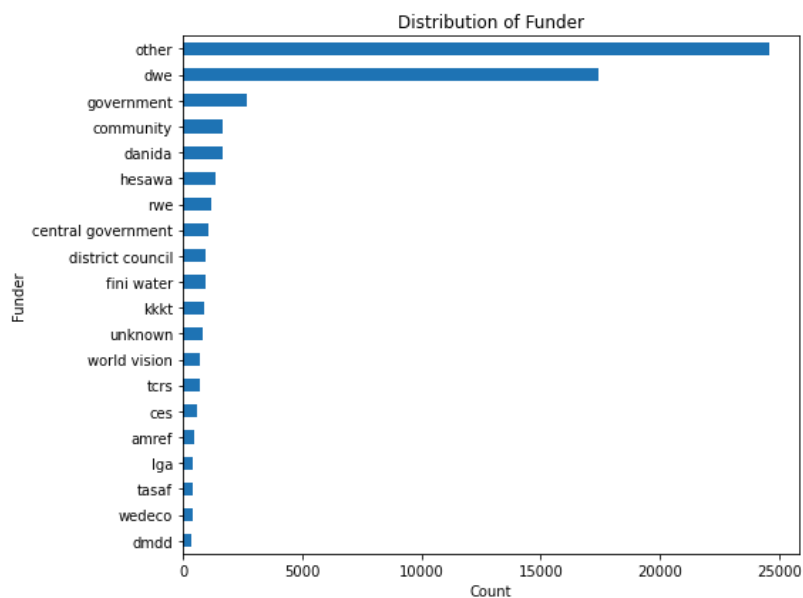
```
In [498]: counts = train_df['funder'].value_counts().sort_values(ascending=True)

plt.figure(figsize=(8, 6))
counts.plot(kind='barh') # horizontal bar chart
plt.title("Distribution of Funder")
plt.xlabel("Count")
plt.ylabel("Funder")
plt.tight_layout()
plt.show()
```



```
In [499]: counts = train_df['installer'].value_counts().sort_values(ascending=True)

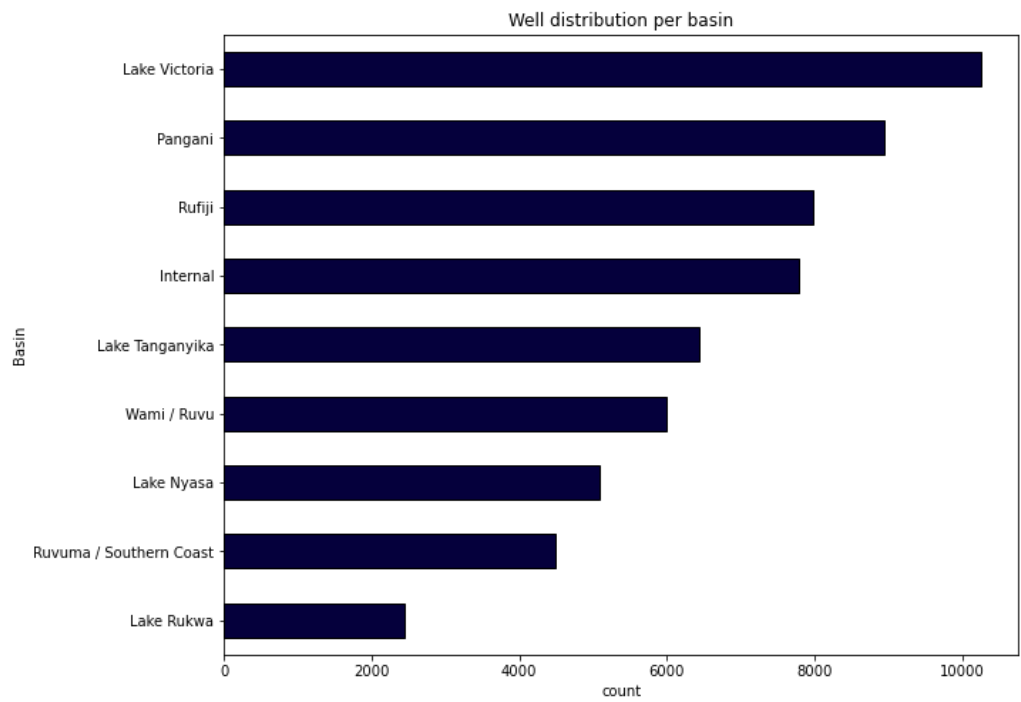
plt.figure(figsize=(8, 6))
counts.plot(kind='barh') # horizontal bar chart
plt.title("Distribution of Funder")
plt.xlabel("Count")
plt.ylabel("Funder")
plt.tight_layout()
plt.show()
```



```
In [500]: # visualizing well distribution per basin
plt.figure(figsize=(10, 8))
basin_counts= train_df["basin"].value_counts().sort_values(ascending= True)

basin_counts.plot(kind= "barh", color= "#090040", edgecolor= "black")
plt.title("Well distribution per basin")
plt.xlabel("count")
plt.ylabel("Basin")

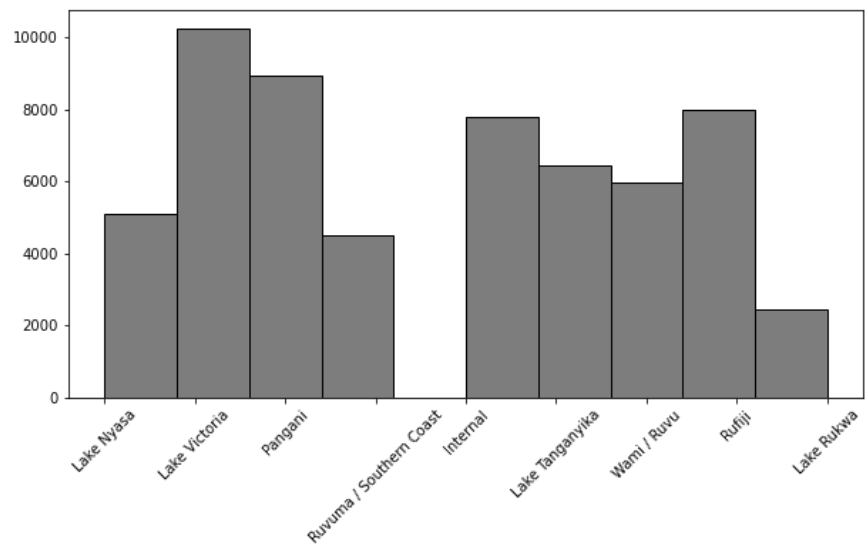
plt.show()
```



Observation:

Lake

```
In [501]: plt.figure(figsize=(10,5))
plt.hist(train_df["basin"], bins= 10, color= "grey", edgecolor= "black")
plt.xticks(rotation= 45)
plt.title("");
```



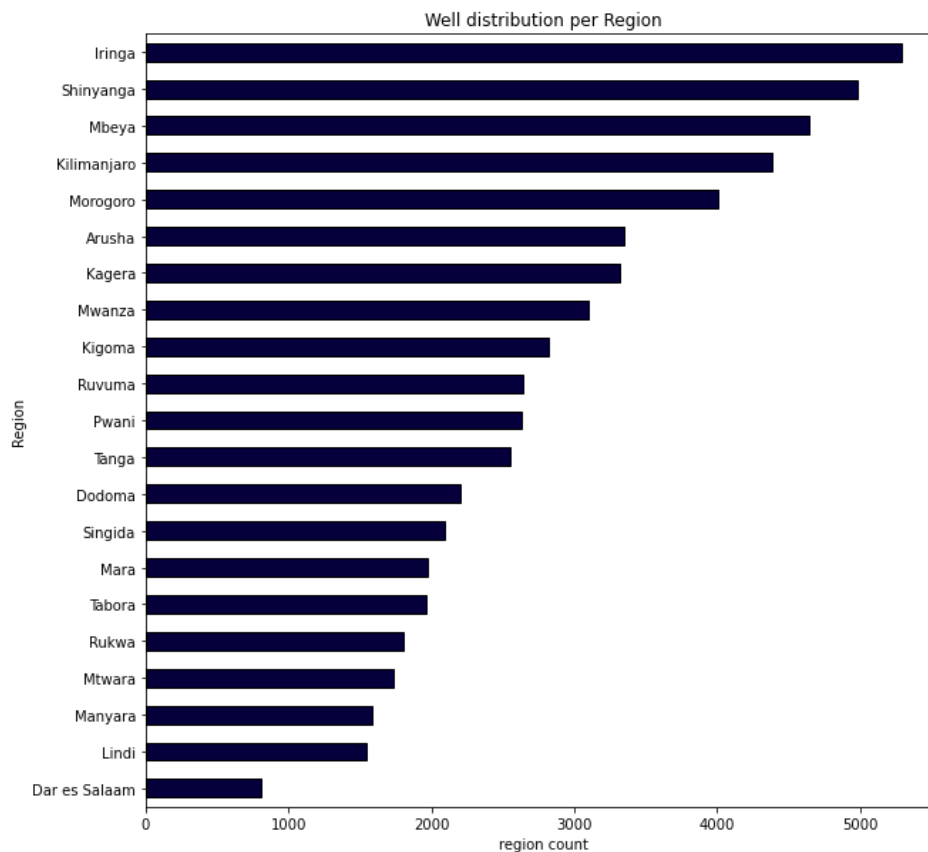
Observation


```
In [502]: # distribution of wells per region

plt.figure(figsize= (10, 10))
region_counts= train_df["region"].value_counts().sort_values(ascending= True)

region_counts.plot(kind= "barh", color= "#090040", edgecolor= "black")
plt.title("Well distribution per Region")
plt.xlabel("region count")
plt.ylabel("Region")

plt.show()
```

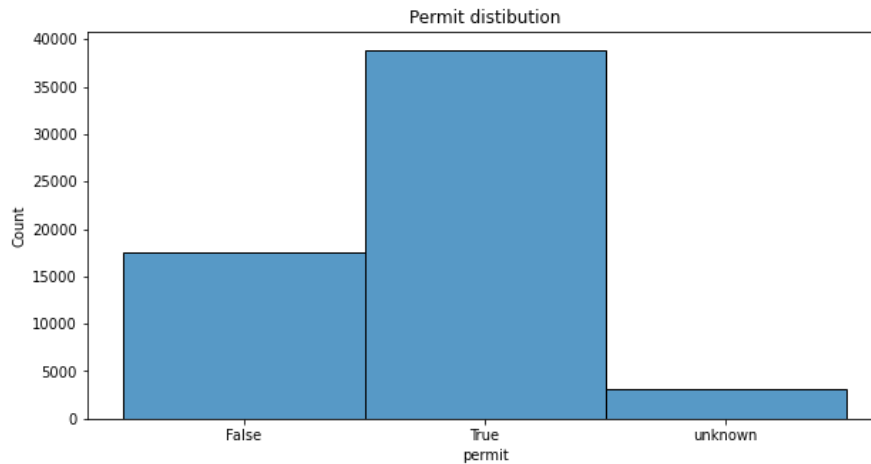


Observation:

The Region with the highest number of wells is Iringa(more than 5000) and the region with the low number of wells is Dar es Salaam(less than 1000)

```
In [503]: # permit distribution
plt.figure(figsize=(10,5))
sns.histplot(train_df["permit"])
plt.title("Permit distribution")
```

Out[503]: Text(0.5, 1.0, 'Permit distribution')



Observation:

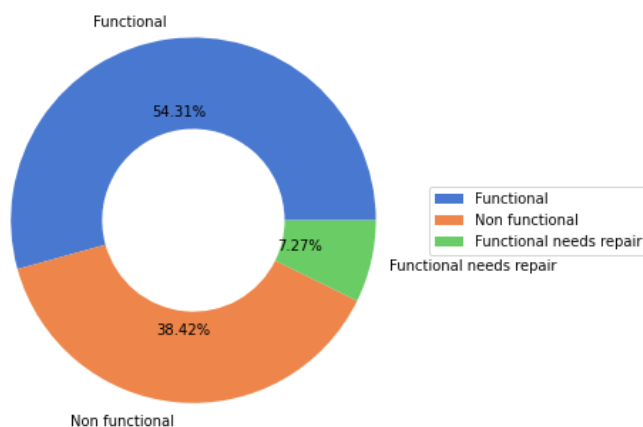
Most(over 35000) of the wells constructed by given permission to operate after completion, others(about 15000) were not granted permission to operationalize, while about 2000 were not known if the permission was given or not

```
In [504]: # visualizing status group distribution
status_value = train_df.status_group.value_counts(ascending=False)

label = ["Functional", "Non functional", "Functional needs repair"]

#plot pie chat
plt.figure(figsize=(10, 5))
color = sns.color_palette('muted')
plt.pie(status_value, labels=label, autopct="%.2f%", wedgeprops=dict(width=.5), colors=color, )
plt.title("Well Status Distribution", fontsize=16, fontweight='bold')
plt.legend(labels=label, loc="center left", bbox_to_anchor=(1, 0, 0.5, 1))
plt.tight_layout();
```

Well Status Distribution



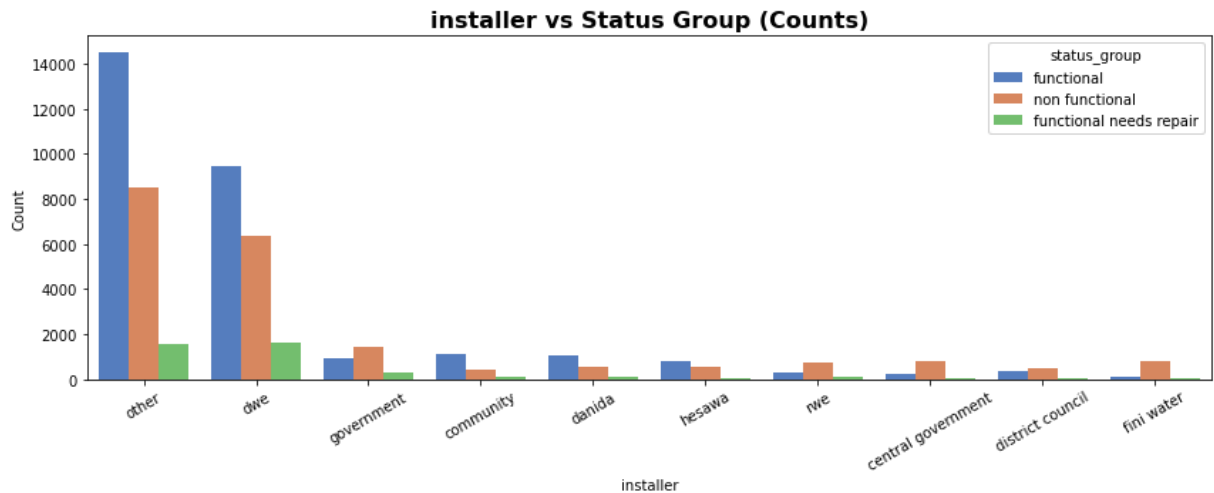
Observation:

- About 54.31% of wells were in good condition
- About 38.42% of the wells were not functional
- About 7.27% of the wells were still functioning but needed repair

Bivariate analysis

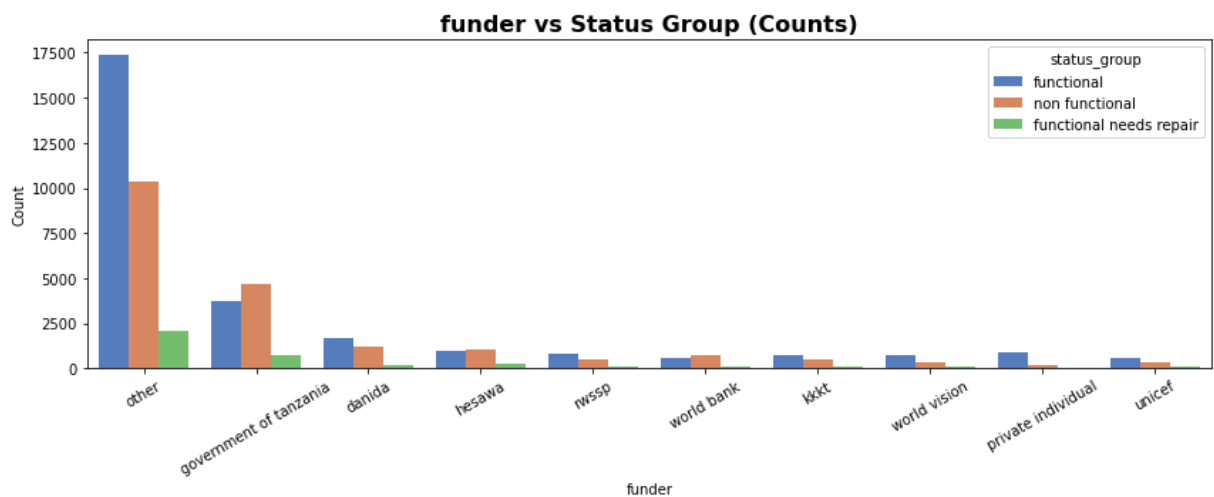
```
In [505]: # visualizing installer vs status group

col = "installer"
plt.figure(figsize=(12, 5))
sns.countplot(x=col, hue='status_group', data=train_df, palette="muted", order=train_df[col].value_counts().index[
plt.title(f"{col} vs Status Group (Counts)", fontsize=16, fontweight='bold')
plt.xlabel(col)
plt.ylabel("Count")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

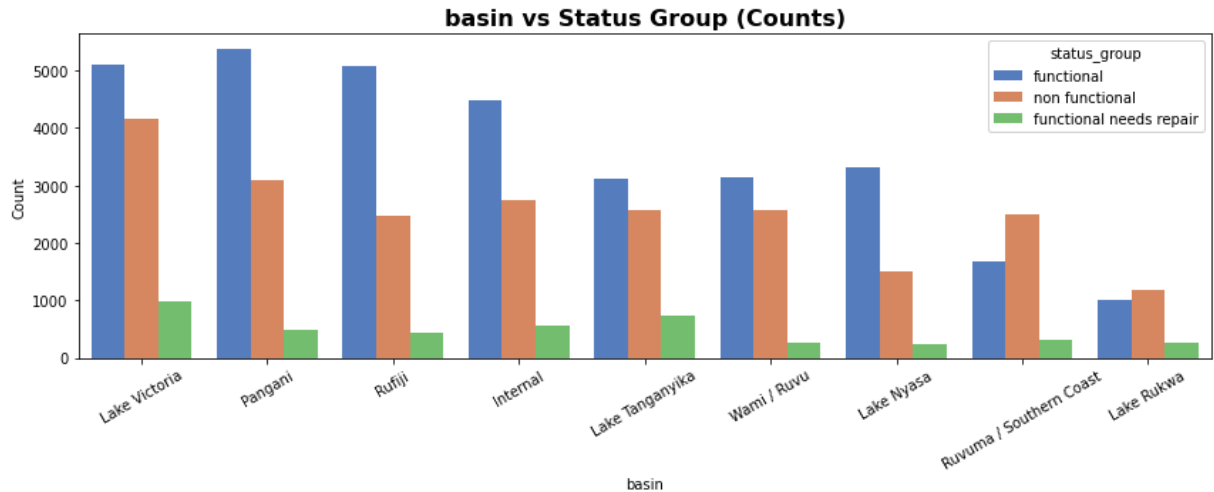


```
In [506]: # visualizing funder vs status group

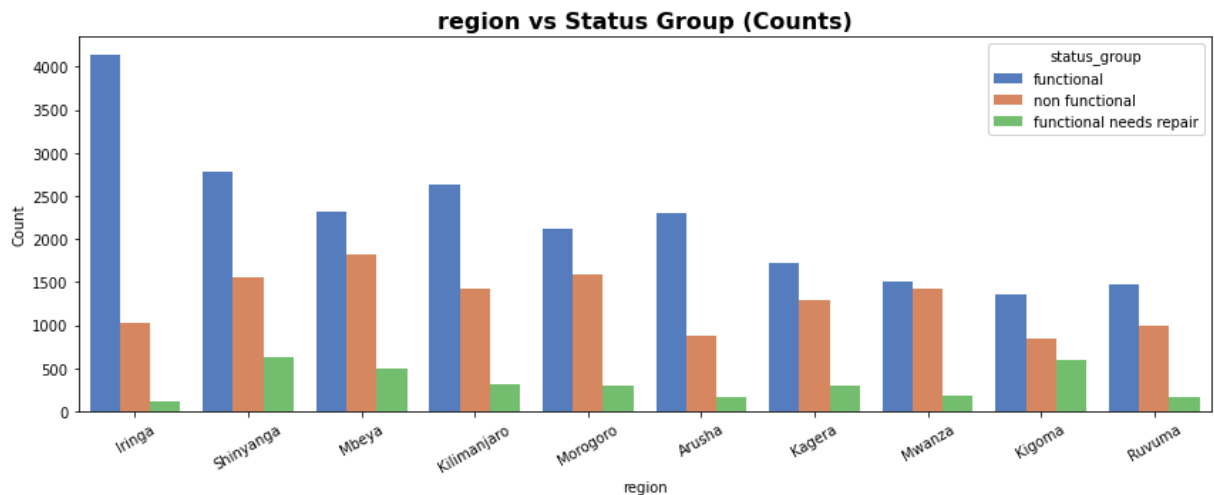
col = "funder"
plt.figure(figsize=(12, 5))
sns.countplot(x=col, hue='status_group', data=train_df, palette="muted", order=train_df[col].value_counts().index[
plt.title(f"{col} vs Status Group (Counts)", fontsize=16, fontweight='bold')
plt.xlabel(col)
plt.ylabel("Count")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```



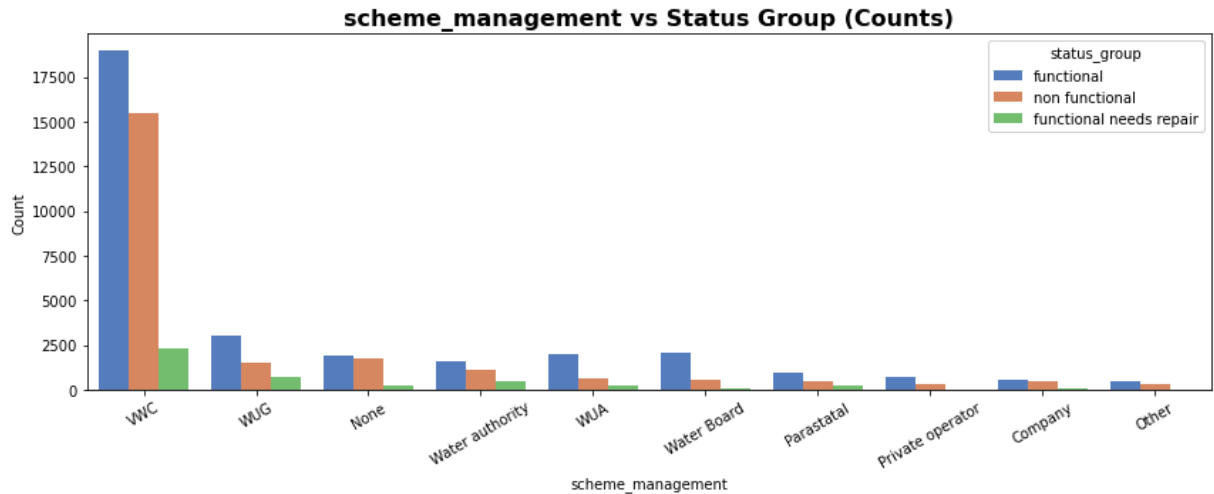
```
In [507]: # visualizing basin by status group
col = "basin"
plt.figure(figsize=(12, 5))
sns.countplot(x=col, hue='status_group', data=train_df, palette="muted", order=train_df[col].value_counts().index)
plt.title(f"{col} vs Status Group (Counts)", fontsize=16, fontweight='bold')
plt.xlabel(col)
plt.ylabel("Count")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```



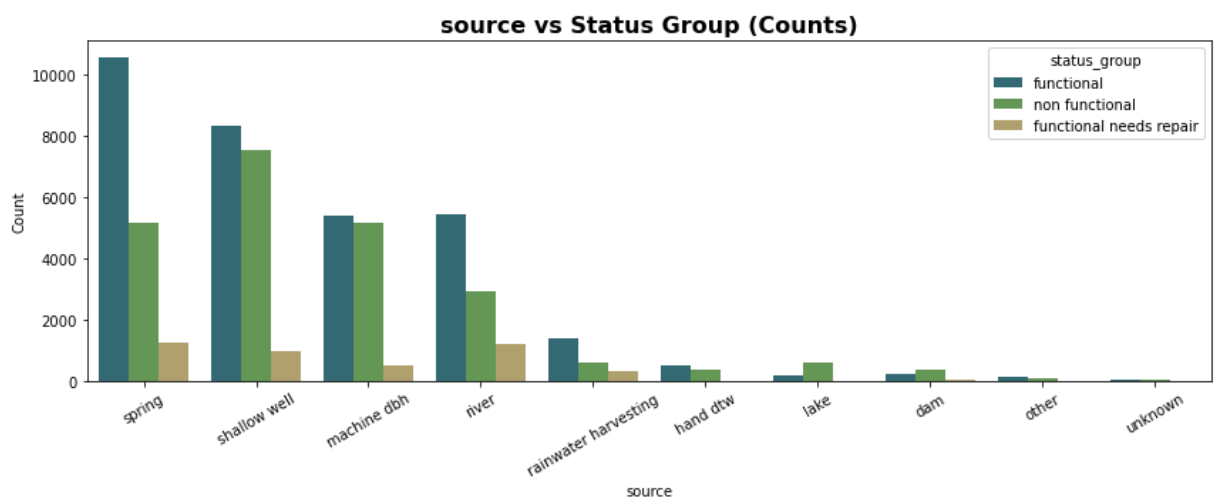
```
In [508]: # visualizing region per status group
col = "region"
plt.figure(figsize=(12, 5))
sns.countplot(x=col, hue='status_group', data=train_df, palette="muted", order=train_df[col].value_counts().index)
plt.title(f"{col} vs Status Group (Counts)", fontsize=16, fontweight='bold')
plt.xlabel(col)
plt.ylabel("Count")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```



```
In [509]: # visualizing scheme management vs status group
col = "scheme_management"
plt.figure(figsize=(12, 5))
sns.countplot(x=col, hue='status_group', data=train_df, palette="muted", order=train_df[col].value_counts().index)
plt.title(f"{col} vs Status Group (Counts)", fontsize=16, fontweight='bold')
plt.xlabel(col)
plt.ylabel("Count")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

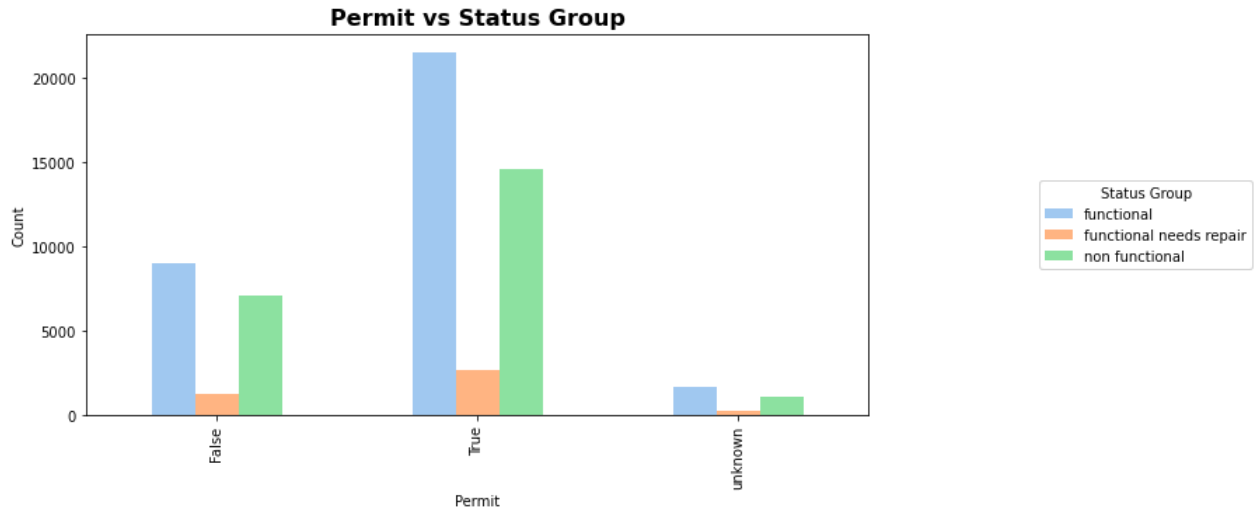


```
In [510]: # visualizing source vs status group
col = "source"
plt.figure(figsize=(12, 5))
sns.countplot(x=col, hue='status_group', data=train_df, palette="gist_earth", order=train_df[col].value_counts().index)
plt.title(f"{col} vs Status Group (Counts)", fontsize=16, fontweight='bold')
plt.xlabel(col)
plt.ylabel("Count")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```



```
In [511]: # visualizing permit vs status
ct = pd.crosstab(train_df["permit"], train_df["status_group"])
plt.figure(figsize=(12,8))
ax = ct.plot(kind="bar", figsize=(10, 5), color=sns.color_palette("pastel"))
plt.title("Permit vs Status Group", fontsize=16, fontweight='bold')
plt.xlabel("Permit")
plt.ylabel("Count")
plt.legend(title="Status Group", loc="center right", bbox_to_anchor=(1, 0, 0.5, 1))
plt.show();
```

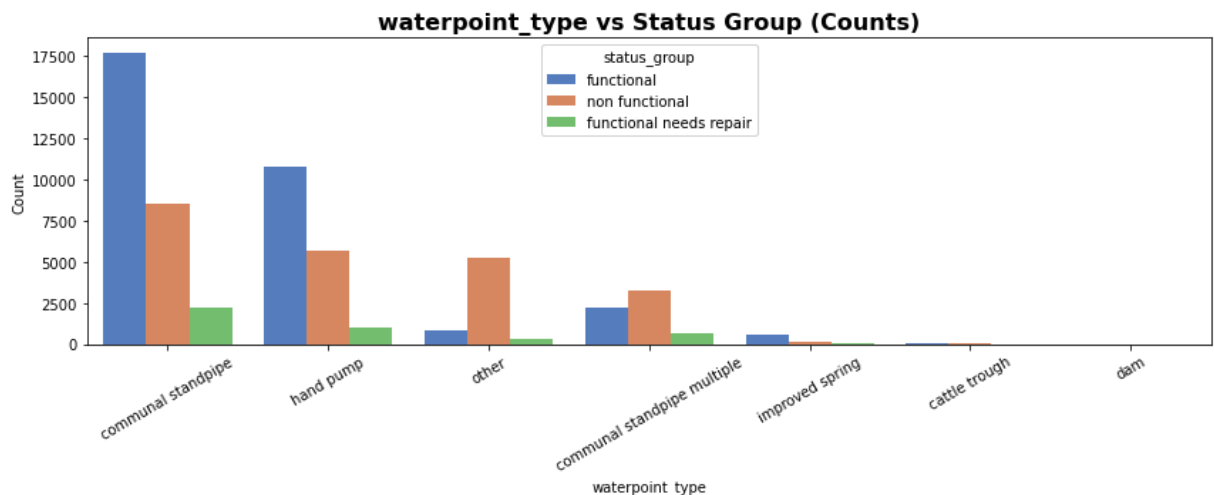
<Figure size 864x576 with 0 Axes>



Observation:

- Wells with permits have the highest number of functional wells as wells as non functional wells

```
In [512]: # waterpoint type vs status group
col = "waterpoint_type"
plt.figure(figsize=(12, 5))
sns.countplot(x=col, hue='status_group', data=train_df, palette="muted", order=train_df[col].value_counts().index)
plt.title(f"{col} vs Status Group (Counts)", fontsize=16, fontweight='bold')
plt.xlabel(col)
plt.ylabel("Count")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show();
```

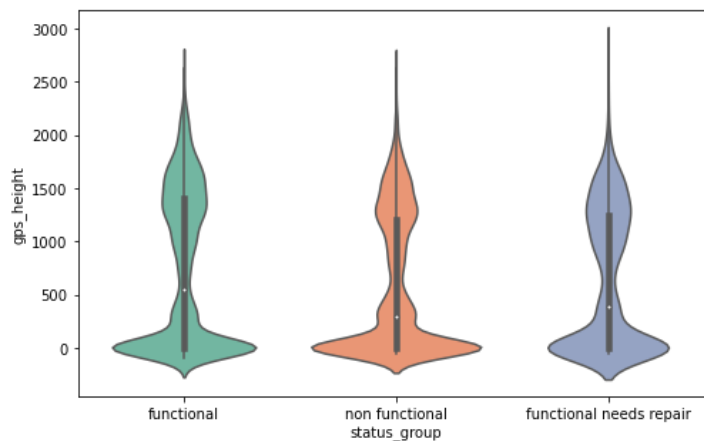


Observation:

- Communal standpipe has the highest number of functional wells as wells as highest number of non functional wells and functional ones that needs repair.

```
In [513]: # visualizing gps height against status group
plt.figure(figsize=(8, 5))
sns.violinplot(
    data=train_df,
    x="status_group",
    y="gps_height",
    palette="Set2")
```

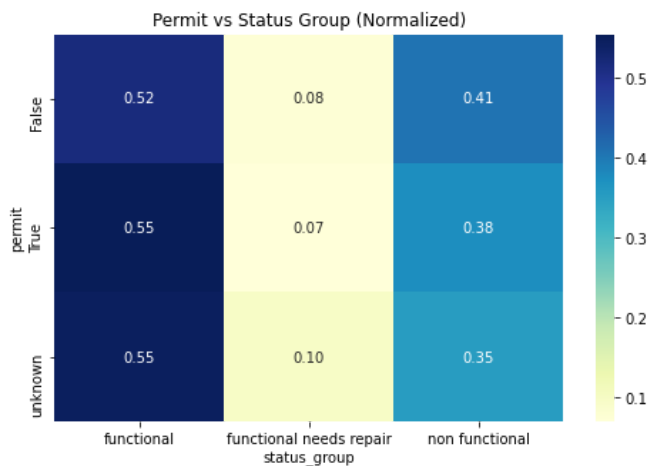
Out[513]: <AxesSubplot:xlabel='status_group', ylabel='gps_height'>



Observation:

This violin plot compares the distribution of `gps_height` across functional, needs-repair, and non-functional waterpoints. The distributions look very similar, with overlapping medians and interquartile ranges, suggesting that elevation does not have a strong effect on waterpoint status. This means `gps_height` might not be a very strong predictor in our model.

```
In [514]: # crosstab showing permit per status group
cross_tab = pd.crosstab(train_df["permit"], train_df["status_group"], normalize="index")
plt.figure(figsize=(8, 5))
sns.heatmap(cross_tab, annot=True, cmap="YlGnBu", fmt=".2f")
plt.title("Permit vs Status Group (Normalized)")
plt.show()
```

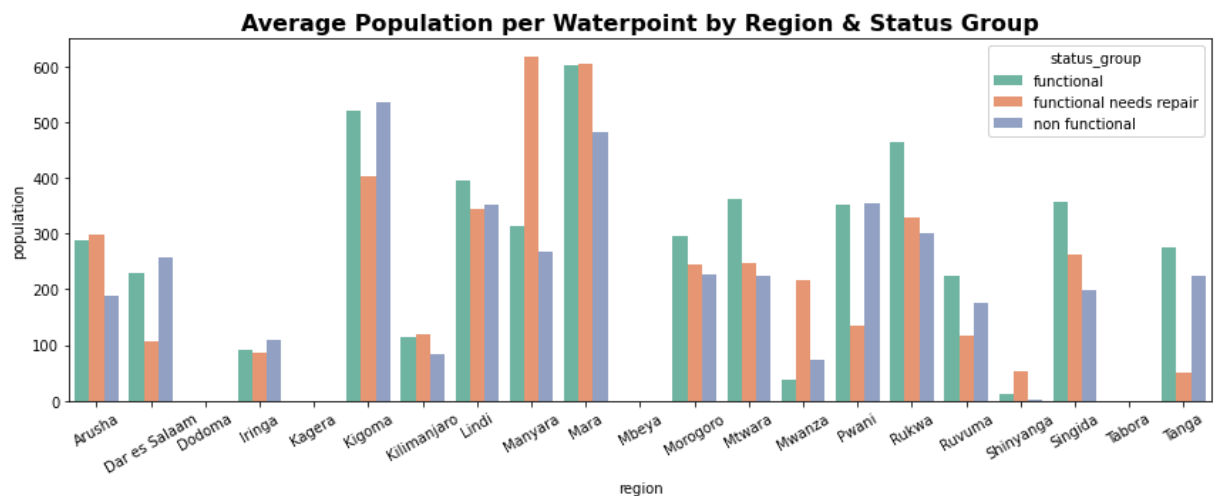


Multivariate analysis

```
In [515]: # visualizing the average population per waterpoint for each region, split by waterpoint status group
agg_df = train_df.groupby(["region", "status_group"])["population"].mean().reset_index()
plt.figure(figsize=(12, 5))
sns.barplot(x="region", y="population", hue="status_group", data=agg_df, palette="Set2")
plt.title("Average Population per Waterpoint by Region & Status Group", fontsize=16, fontweight="bold")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

for p in ax.patches:
    height = p.get_height()
    ax.annotate(f'{height:.0f}',
                (p.get_x() + p.get_width() / 2., height),
                ha='center', va='bottom', fontsize=8, rotation=0)

plt.show()
```



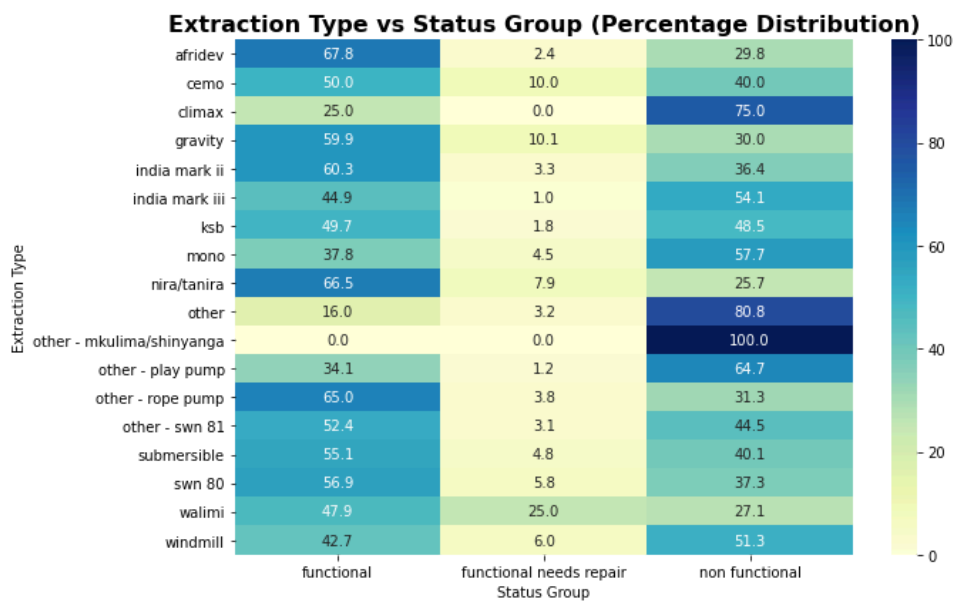
Observation:

The bar graph above shows the average population served by waterpoints across different regions and status group.

Areas like mara serve larger population per waterpoint. and area like Manyara has functional well that needs repair but it serves large population.


```
In [516]: # Creating a crosstab and normalizing by row to get percentages
cross_tab = pd.crosstab(train_df["extraction_type"], train_df["status_group"], normalize="index") * 100

plt.figure(figsize=(10, 6))
sns.heatmap(cross_tab, annot=True, cmap="YlGnBu", fmt=".1f")
plt.title("Extraction Type vs Status Group (Percentage Distribution)", fontsize=16, fontweight="bold")
plt.ylabel("Extraction Type")
plt.xlabel("Status Group")
plt.tight_layout()
plt.show()
```



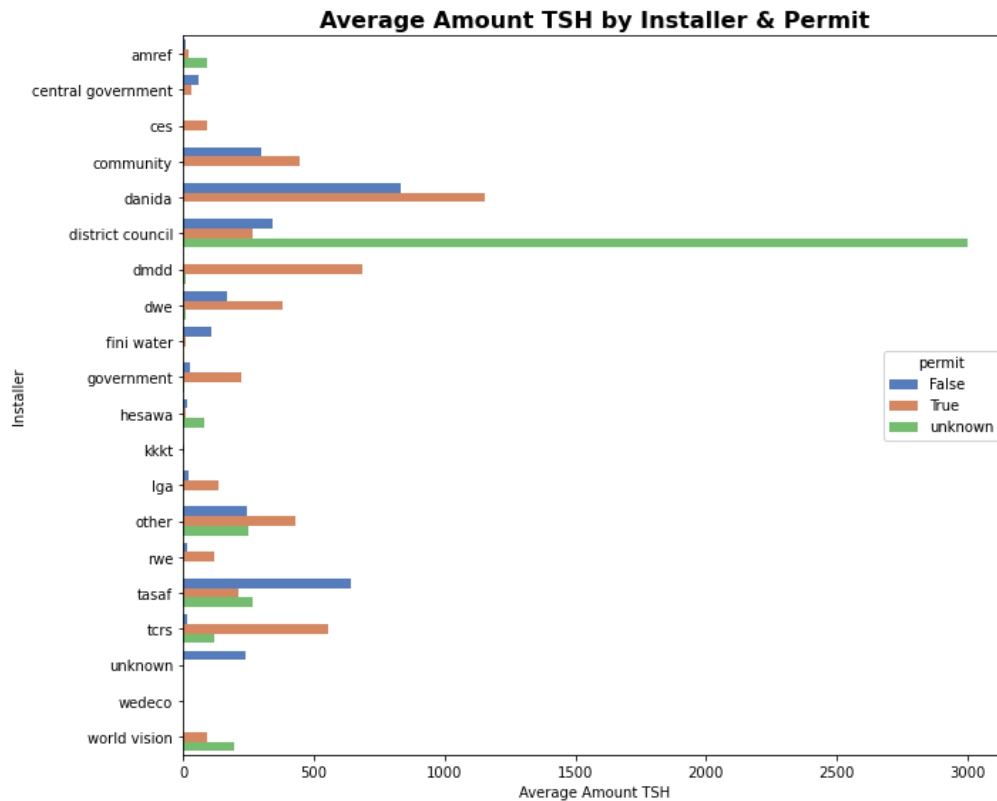
```

In [518]: # plotting Average Amount TSH by Installer & Permit
agg_df = train_df.groupby(["installer", "permit"])["amount_tsh"].mean().reset_index()

plt.figure(figsize=(10, 8))
sns.barplot(
    y="installer",
    x="amount_tsh",
    hue="permit",
    data=agg_df,
    orient="h",
    palette="muted"
)

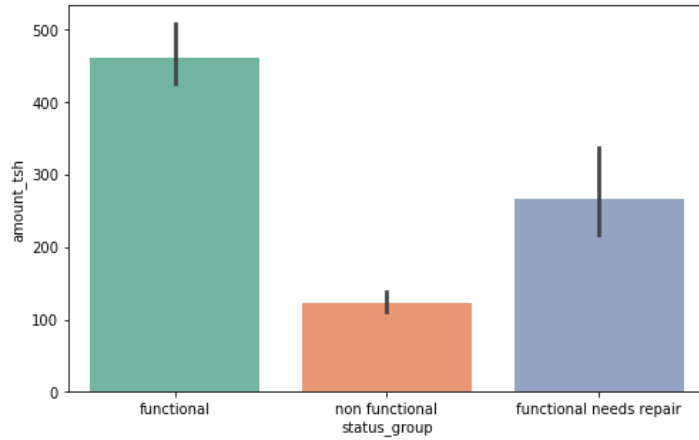
plt.title("Average Amount TSH by Installer & Permit", fontsize=16, fontweight="bold")
plt.xlabel("Average Amount TSH")
plt.ylabel("Installer")
plt.tight_layout()
plt.show()

```



```
In [519]: # plotting amount in total static head of a well per status group
plt.figure(figsize=(8, 5))
sns.barplot(
    data=train_df,
    x="status_group",
    y="amount_tsh",
    palette="Set2"
)
```

Out[519]: <AxesSubplot:xlabel='status_group', ylabel='amount_tsh'>



```
In [521]: # Combining extraction_type + waterpoint_type
train_df["extract_waterpoint"] = (
    train_df["extraction_type"].astype(str) + "_" + train_df["waterpoint_type"].astype(str)
)

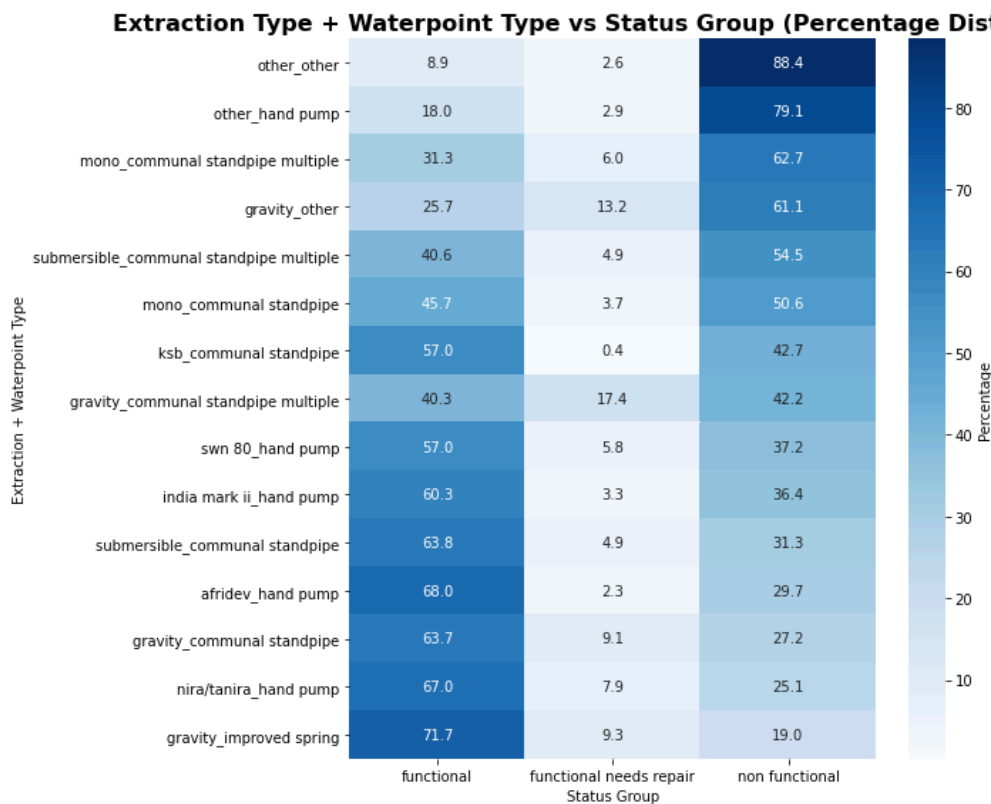
# Getting the top 15 most frequent combinations
top_combos = train_df["extract_waterpoint"].value_counts().head(15).index
filtered_df = train_df[train_df["extract_waterpoint"].isin(top_combos)]

# Create a crosstab: row = combination, col = status_group, normalize row-wise to get %
cross_tab = pd.crosstab(
    filtered_df["extract_waterpoint"],
    filtered_df["status_group"],
    normalize="index"
) * 100

# Sorting by percentage of non functional (helps to see worst combos first)
cross_tab = cross_tab.sort_values("non functional", ascending=False)

#Plotting the heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(
    cross_tab,
    annot=True,
    fmt=".1f",
    cmap="Blues",
    cbar_kws={'label': 'Percentage'}
)

plt.title("Extraction Type + Waterpoint Type vs Status Group (Percentage Distribution)",
        fontsize=16, fontweight="bold")
plt.xlabel("Status Group")
plt.ylabel("Extraction + Waterpoint Type")
plt.tight_layout()
plt.show()
```



Observation:

- This heatmap visualizes the percentage distribution of each extraction_type + waterpoint_type combination across the three status_group categories.
- Rows with a very high percentage in 'non functional' highlight problematic combinations that are strongly associated with failure. Similarly, combinations with high 'functional' percentages represent reliable setups.

- Focuses only on the most common combinations that is removes noise from rare cases

```
In [522]: # saving a csv file
train_df.to_csv("Cleaned_wells_df", index = False)
```

Modeling

Model preprocessing

```
In [523]: data= pd.read_csv("Cleaned_wells_df")
data.head()
```

Out[523]:

	amount_tsh	funder	gps_height	installer	longitude	latitude	basin	region	lga	population	...	construction_year	extraction
0	6000	roman catholic	1390	other	34.938093	-9.856322	Lake Nyasa	Iringa	Ludewa	109	...	1999.0	g
1	0	other	1399	other	34.698766	-2.147466	Lake Victoria	Mara	Serengeti	280	...	2010.0	g
2	25	other	686	world vision	37.460664	-3.821329	Pangani	Manyara	Simanjiro	250	...	2009.0	g
3	0	unicef	263	other	38.486161	-11.155298	Ruvuma / Southern Coast	Mtwara	Nanyumbu	58	...	1986.0	subme
4	0	other	0	other	31.130847	-1.825359	Lake Victoria	Kagera	Karagwe	0	...	2000.0	g

5 rows × 23 columns

```
In [524]: # Seperate labels from features
X= data.drop("status_group", axis=1)
y= data.status_group
```

```
In [525]: # splitting the data before encoding and fit transforming to avoid data Leakage
X_train, X_test, y_train, y_test= train_test_split(X, y, test_size= .3, random_state= 42)
```

```
In [526]: # checking shapes of the splitted data
print("X_train_shape=", X_train.shape)
print("X_test_shape=", X_test.shape)
print("y_train_shape=", y_train.shape)
print("y_test_shape=", y_test.shape)
```

```
X_train_shape= (41580, 22)
X_test_shape= (17820, 22)
y_train_shape= (41580,)
y_test_shape= (17820,)
```

```
In [527]: # standardizing the numerical columns before encoding the the categorical columns
numeric_std_cols= X_train.select_dtypes(include= "number").columns
numeric_std_cols

# scaling the numerical cols
ss = StandardScaler()
X_train_s =ss.fit_transform(X_train[numeric_std_cols])
X_test_s =ss.transform(X_test[numeric_std_cols])
```

```
In [528]: # combining scaled data into a dataframe
X_train_scaled= pd.DataFrame(X_train_s, columns= numeric_std_cols, index= X_train.index)
X_test_scaled= pd.DataFrame(X_test_s, columns= numeric_std_cols, index= X_test.index)
```

```
In [529]: # One hot encoding the categorical columns of the dataframes
ohe= OneHotEncoder(handle_unknown='ignore', sparse_output=False)

# categorical columns in Xtrain data

cat_cols_train= X_train.select_dtypes(include= "object").columns
# cat cols in Xtest data
cat_cols_test= X_test.select_dtypes(include= "object").columns

# fitting train data
ohe.fit(X_train[cat_cols_train])

# Transforming both train and test into dataframes
X_train_encoded= pd.DataFrame(ohe.transform(X_train[cat_cols_train]), columns= ohe.get_feature_names_out(cat_cols_
index= X_train.index)

X_test_encoded= pd.DataFrame(ohe.transform(X_test[cat_cols_test]), columns= ohe.get_feature_names_out(cat_cols_tes
index= X_test.index)
```

```
In [530]: # concating both scaled num daframes and ecoded dataframes in one final dataset that is scaled and onehotencoded
X_train_final= pd.concat([X_train_encoded, X_train_scaled], axis= 1)
X_test_final= pd.concat([X_test_encoded, X_test_scaled], axis= 1)
```

```
In [531]: # encoding the target variable
if y_train.dtype == "object":
    label_encoder= LabelEncoder()
    y_train= label_encoder.fit_transform(y_train)
    y_test= label_encoder.transform(y_test)
```

```
In [532]: # checking the shapes of the dataframes before scaling and encoding
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
print("y_train class distribution:", np.bincount(y_train)) # Checking class balance

X_train shape: (41580, 22)
X_test shape: (17820, 22)
y_train shape: (41580,)
y_test shape: (17820,)
y_train class distribution: [22535 3024 16021]
```

```
In [533]: # checking the combined dataframe shapes after scaling and encoding
print("X_train_final shape:", X_train_final.shape)
print("X_test_final shape:", X_test_final.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
print("y_train class distribution:", np.bincount(y_train)) # Checking class balance

X_train_final shape: (41580, 352)
X_test_final shape: (17820, 352)
y_train shape: (41580,)
y_test shape: (17820,)
y_train class distribution: [22535 3024 16021]
```

Observation:

- Its clearly notable that is a class imbalance problem whereby the mainority class (functional needs repair 3024) is much smaller then the others which may cause the model to favour majority of the classes(functional and non functional)
- will use SMOTE to resolve the problem

Logistic regression model

baseline model

```
In [534]: # creating a logistic regression model
logreg= LogisticRegression(random_state= 42, multi_class= 'multinomial', max_iter= 1000)
logreg
```

```
Out[534]: LogisticRegression
LogisticRegression(max_iter=1000, multi_class='multinomial', random_state=42)
```

```
In [535]: # fit training the model
logreg.fit(X_train_final, y_train)
```

```
Out[535]: LogisticRegression
LogisticRegression(max_iter=1000, multi_class='multinomial', random_state=42)
```

```
In [536]: print(f" the model score on training dataset is {logreg.score(X_train_final, y_train)*100:.2f}%")
```

the model score on training dataset is 75.84%

```
In [537]: #checking prediction and accuracy

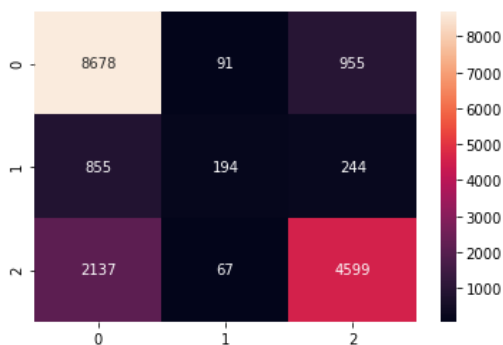
y_pred = logreg.predict(X_test_final)

#accuracy
accuracy_score(y_test, y_pred)*100
```

```
Out[537]: 75.59483726150393
```

```
In [538]: # confusion matrix
conf = confusion_matrix(y_test, y_pred)
sns.heatmap(conf, annot=True, fmt= "d")
```

```
Out[538]: <AxesSubplot:>
```



```
In [539]: # classification report
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.74	0.89	0.81	9724
1	0.55	0.15	0.24	1293
2	0.79	0.68	0.73	6803
accuracy			0.76	17820
macro avg	0.70	0.57	0.59	17820
weighted avg	0.75	0.76	0.74	17820

Smote with logistic regression model

```
In [540]: # distribution of classes before smoting
pd.Series(y_train).value_counts(1)
```

```
Out[540]: 0    0.541967
          2    0.385305
          1    0.072727
          dtype: float64
```

```
In [541]: # smoting dataframes
smote= SMOTE(random_state= 42)

X_train_smote, y_train_smote = smote.fit_resample(X_train_final, y_train)
```

```
In [542]: # turning y_train_smote into a series
y_train_smote= pd.Series(y_train_smote)

# Distributuion of classes after SMOTE
y_train_smote.value_counts(1)
```

```
Out[542]: 2    0.333333
          1    0.333333
          0    0.333333
          dtype: float64
```

```
In [543]: # Turning the SMOTE X into a dataframe
X_train_smote = pd.DataFrame(X_train_smote, columns=X_train_final.columns)
X_train_smote.head()
```

```
Out[543]:
```

	funder_danida	funder_dhv	funder_district council	funder_dwsp	funder_fini water	funder_german republic	funder_government of tanzania	funder_hesawa	funder_kkkt	funder
0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	
4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	

5 rows × 352 columns

```
In [544]: logreg1= LogisticRegression(random_state= 42, multi_class= 'multinomial', max_iter= 1000)

# fitting the model
logreg1.fit(X_train_smote, y_train_smote)
```

```
Out[544]: LogisticRegression
LogisticRegression(max_iter=1000, multi_class='multinomial', random_state=42)
```

```
In [545]: print(f" the model score on training dataset is {logreg.score(X_train_smote, y_train_smote)*100:.2f}%")

the model score on training dataset is 57.70%
```

```
In [546]: #checking prediction and accuracy

y_pred = logreg1.predict(X_test_final)

#accuracy
accuracy_score(y_test, y_pred)*100
```

```
Out[546]: 65.48260381593714
```

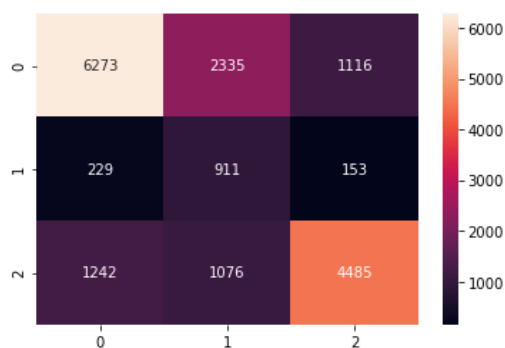


```
In [547]: # classification report
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.81	0.65	0.72	9724
1	0.21	0.70	0.32	1293
2	0.78	0.66	0.71	6803
accuracy			0.65	17820
macro avg	0.60	0.67	0.59	17820
weighted avg	0.75	0.65	0.69	17820

```
In [548]: # confusion matrix
conf = confusion_matrix(y_test, y_pred)
sns.heatmap(conf, annot=True, fmt="d")
```

Out[548]: <AxesSubplot:>



```
In [549]: # classification report
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.81	0.65	0.72	9724
1	0.21	0.70	0.32	1293
2	0.78	0.66	0.71	6803
accuracy			0.65	17820
macro avg	0.60	0.67	0.59	17820
weighted avg	0.75	0.65	0.69	17820

Observation:

The model performs reasonably well on functional and non-functional wells. Handling class imbalance using SMOTE, the minority class recall(functioning needs repair, 1) score has improved from 15 % to 70%.

In this model we will major heavily on recall, because its much worse for non functional wells to be predicted to be functional than for functional wells to be predicted to be non functional. therefore it is better to predict functional wells as non functional because if non functional wells are not correctly predicted, people will potentially be left without water source.

```
In [*]: # Getting predicted probabilities for each class
y_score = logreg1.predict_proba(X_train_smote)

# Binarizing the y Labels for ROC (One-vs-Rest)
classes = np.unique(y_train_smote)
y_bin = label_binarize(y_train_smote, classes=classes)
```

```
In [551]: # computing ROC and AUC curve for each class
fpr = dict()
tpr = dict()
roc_auc = dict()

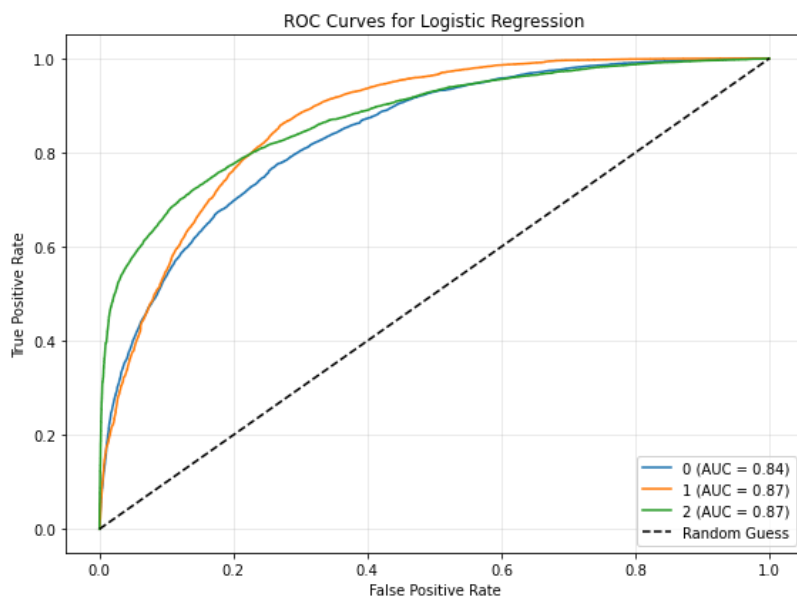
for i, cls in enumerate(classes):
    fpr[i], tpr[i], thresholds = roc_curve(y_bin[:, i], y_score[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])
```

```
In [552]: plt.figure(figsize=(8,6))

for i, cls in enumerate(classes):
    plt.plot(fpr[i], tpr[i], label=f"{cls} (AUC = {roc_auc[i]:.2f})")

# Adding diagonal line (random classifier)
plt.plot([0, 1], [0, 1], "k--", label="Random Guess")

plt.title("ROC Curves for Logistic Regression")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



Observation:

The logistic regression model achieved strong classification performance across all three categories. The ROC–AUC scores were 0.84 for functional wells, 0.87 for wells needing repair, and 0.87 for non-functional wells. This indicates that the model is particularly effective at distinguishing problematic wells (needing repair or non-functional) from functional ones, which is valuable for prioritizing maintenance interventions.

Elastic Net Model

```
In [*]: # Creating the classifier
logreg_eln= LogisticRegression(solver = 'saga', penalty= "elasticnet", multi_class= "multinomial", l1_ratio= 0.5,
                               max_iter= 1000, random_state= 42)

# Fitting to the training data
elastic_logreg = logreg_eln.fit(X_train_smote, y_train_smote)

# Predicting the y elastic reg
y_elastic_preds = elastic_logreg.predict(X_test_final)

# accuracy score
accuracy_score(y_test, y_pred)*1000

# evaluation
print(classification_report(y_test, y_elastic_preds))
```

```
In [582]: # Getting predicted probabilities for each class
y_score_elastic = logreg_eln.predict_proba(X_test_final)

# Binarizing the y Labels for ROC (One-vs-Rest)
classes = np.unique(y_test)
y_bin_rf = label_binarize(y_test, classes=classes)

# computing ROC and AUC curve for each class
fpr = dict()
tpr = dict()
roc_auc = dict()

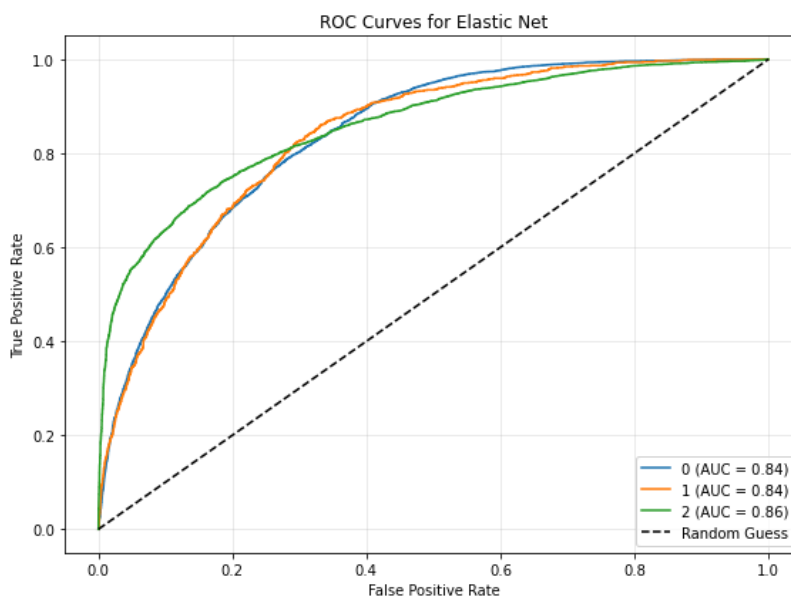
for i, cls in enumerate(classes):
    fpr[i], tpr[i], thresholds = roc_curve(y_bin_rf[:, i], y_score_elastic[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

plt.figure(figsize=(8,6))

for i, cls in enumerate(classes):
    plt.plot(fpr[i], tpr[i], label=f"{cls} (AUC = {roc_auc[i]:.2f})")

# Adding diagonal line (random classifier)
plt.plot([0, 1], [0, 1], "k--", label="Random Guess")

plt.title("ROC Curves for Elastic Net")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



Random Forest model

Baseline

```
In [583]: # initializing the model
random_forest = RandomForestClassifier(random_state=42)
random_forest
```

```
Out[583]: RandomForestClassifier
RandomForestClassifier(random_state=42)
```

```
In [584]: # fitting the model
rf_model = random_forest.fit(X_train_smote, y_train_smote)
```

```
In [585]: #checking prediction and accuracy

y_rf_model = rf_model.predict(X_test_final)

#accuracy
accuracy_score(y_test, y_rf_model)*100
```

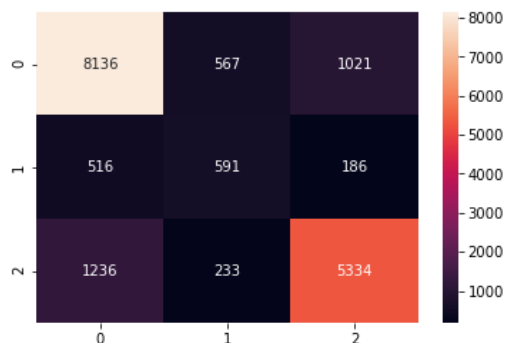
```
Out[585]: 78.9057239057239
```

```
In [586]: # evaluation
print(classification_report(y_test, y_rf_model))
```

	precision	recall	f1-score	support
0	0.82	0.84	0.83	9724
1	0.42	0.46	0.44	1293
2	0.82	0.78	0.80	6803
accuracy			0.79	17820
macro avg	0.69	0.69	0.69	17820
weighted avg	0.79	0.79	0.79	17820

```
In [587]: # confusion table
conf = confusion_matrix(y_test, y_rf_model)
sns.heatmap(conf, annot=True, fmt="d")
```

```
Out[587]: <AxesSubplot:>
```



```
In [588]: # Getting predicted probabilities for each class
y_score_rf = rf_model.predict_proba(X_test_final)

# Binarizing the y Labels for ROC (One-vs-Rest)
classes = np.unique(y_test)
y_bin_rf = label_binarize(y_test, classes=classes)
```

```
In [589]: # computing ROC and AUC curve for each class
fpr = dict()
tpr = dict()
roc_auc = dict()

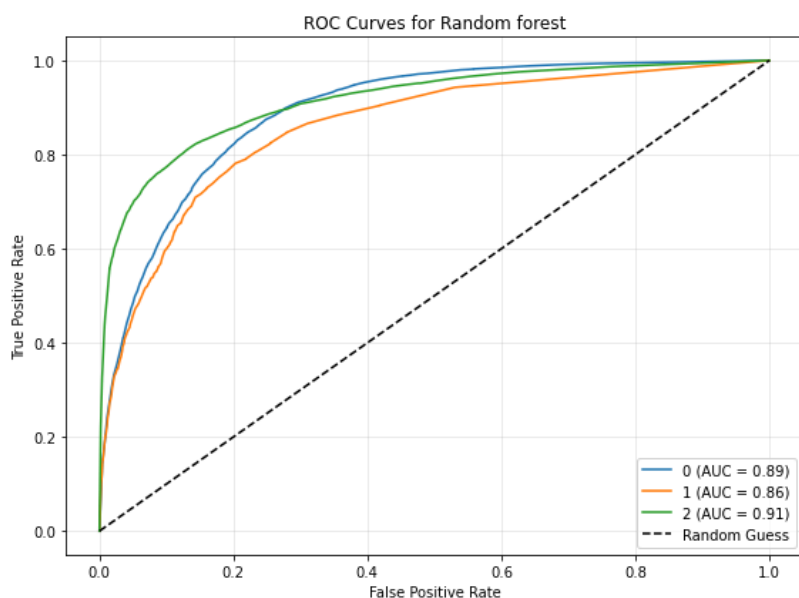
for i, cls in enumerate(classes):
    fpr[i], tpr[i], thresholds = roc_curve(y_bin_rf[:, i], y_score_rf[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])
```

```
In [590]: plt.figure(figsize=(8,6))

for i, cls in enumerate(classes):
    plt.plot(fpr[i], tpr[i], label=f"{cls} (AUC = {roc_auc[i]:.2f})")

# Adding diagonal line (random classifier)
plt.plot([0, 1], [0, 1], "k--", label="Random Guess")

plt.title("ROC Curves for Random forest")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



RandomizedSearchCV

```
In [564]: # hyperparameter tuning

param_dist = {
    "n_estimators": [100, 200, 300],
    "max_depth": [None, 10, 20],
    "min_samples_split": [2, 5, 10]
}

rf_random = RandomizedSearchCV(
    RandomForestClassifier(class_weight="balanced", random_state=42, n_jobs=-1),
    param_distributions=param_dist,
    n_iter=10,
    scoring="recall_weighted",
    cv=3,
    random_state=42,
    n_jobs=-1
)

rf_random.fit(X_train_smote, y_train_smote)
print(f"Best RF Params (Randomized Search): {rf_random.best_params_}")
```

Best RF Params (Randomized Search): {'n_estimators': 100, 'min_samples_split': 2, 'max_depth': None}

```
In [571]: # Use the best estimator found by RandomizedSearchCV
best_rf_model = rf_random.best_estimator_

# Make predictions on the test set
y_pred = best_rf_model.predict(X_test_final)

print("Classification Report on Test Data:")
print(classification_report(y_test, y_pred))
```

Classification Report on Test Data:

	precision	recall	f1-score	support
0	0.82	0.84	0.83	9724
1	0.42	0.46	0.44	1293
2	0.82	0.78	0.80	6803
accuracy			0.79	17820
macro avg	0.69	0.69	0.69	17820
weighted avg	0.79	0.79	0.79	17820

```
In [591]: # Getting predicted probabilities for each class
y_score_random = rf_random.predict_proba(X_test_final)

# Binarizing the y Labels for ROC (One-vs-Rest)
classes = np.unique(y_test)
y_bin_rf = label_binarize(y_test, classes=classes)

# computing ROC and AUC curve for each class
fpr = dict()
tpr = dict()
roc_auc = dict()

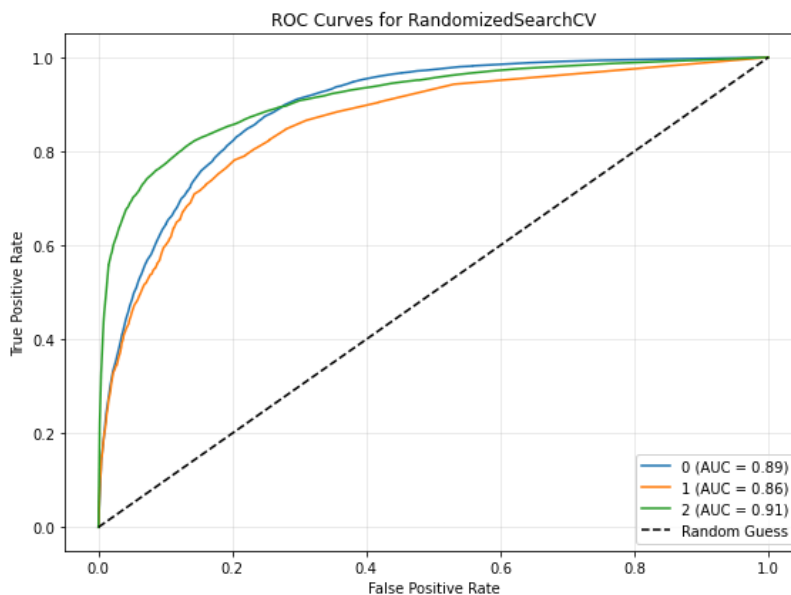
for i, cls in enumerate(classes):
    fpr[i], tpr[i], thresholds = roc_curve(y_bin_rf[:, i], y_score_random[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

plt.figure(figsize=(8,6))

for i, cls in enumerate(classes):
    plt.plot(fpr[i], tpr[i], label=f"{cls} (AUC = {roc_auc[i]:.2f})")

# Adding diagonal Line (random classifier)
plt.plot([0, 1], [0, 1], "k--", label="Random Guess")

plt.title("ROC Curves for RandomizedSearchCV")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



XGBoost model**baseline model(before tuning)**

```
In [566]: # initializing the model
boost= XGBClassifier(random_state= 42)

# fitting the Xtrain and y train
boost.fit(X_train_final, y_train)
```

```
Out[566]: XGBClassifier
XGBClassifier(base_score=0.5, booster='gbtree', colsample_bylevel=1,
              colsample_bynode=1, colsample_bytree=1, gamma=0, gpu_id=-1,
              importance_type='gain', interaction_constraints='',
              learning_rate=0.300000012, max_delta_step=0, max_depth=6,
              min_child_weight=1, missing=nan, monotone_constraints=()),
              n_estimators=100, n_jobs=0, num_parallel_tree=1,
              objective='multi:softprob', random_state=42, reg_alpha=0,
              reg_lambda=1, scale_pos_weight=None, subsample=1,
              tree_method='exact', validate_parameters=1, verbosity=None)
```

```
In [567]: # creating prediction
y_boost_pred= boost.predict(X_test_final)

# accuracy score
accuracy_score(y_test, y_boost_pred)* 100
```

```
Out[567]: 79.18630751964085
```

```
In [572]: print(classification_report(y_test, y_boost_pred))
```

	precision	recall	f1-score	support
0	0.77	0.92	0.84	9724
1	0.66	0.24	0.35	1293
2	0.85	0.72	0.78	6803
accuracy			0.79	17820
macro avg	0.76	0.62	0.66	17820
weighted avg	0.79	0.79	0.78	17820

Observation:

The classification report reveals that the XGBoost model is very good at identifying functional and non-functional wells but is performing very poorly on the "needs repair" class. This is a key weakness that needs to be addressed.

```

In [580]: # Getting predicted probabilities for each class
y_score_boost = boost.predict_proba(X_test_final)

# Binarizing the y Labels for ROC (One-vs-Rest)
classes = np.unique(y_test)
y_bin_rf = label_binarize(y_test, classes=classes)

# computing ROC and AUC curve for each class
fpr = dict()
tpr = dict()
roc_auc = dict()

for i, cls in enumerate(classes):
    fpr[i], tpr[i], thresholds = roc_curve(y_bin_rf[:, i], y_score_boost[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

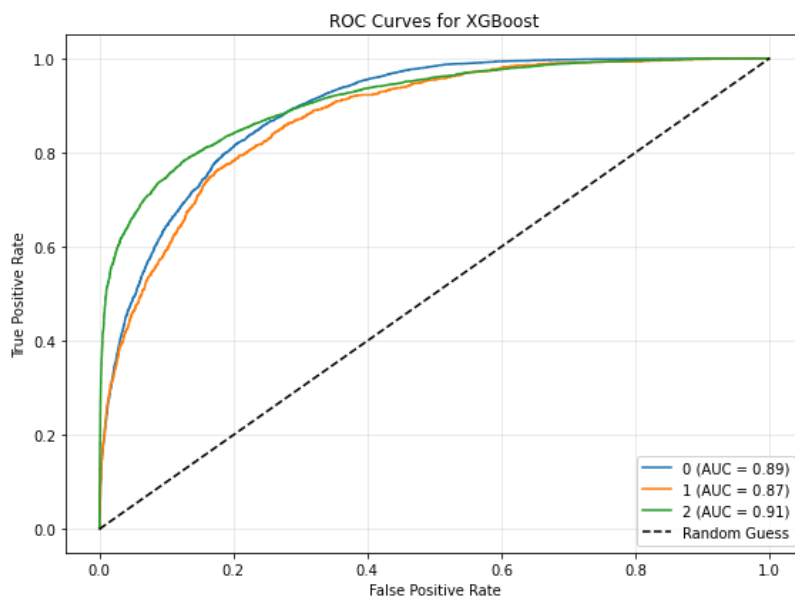
plt.figure(figsize=(8,6))

for i, cls in enumerate(classes):
    plt.plot(fpr[i], tpr[i], label=f"{cls} (AUC = {roc_auc[i]:.2f})")

# Adding diagonal line (random classifier)
plt.plot([0, 1], [0, 1], "k--", label="Random Guess")

plt.title("ROC Curves for XGBoost")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```



```

In [ ]: Observation:

The ROC values of the baseline model look good and improved from the previous Elastic net with logistic regression

```

hyperparameter tuning with RandomizedSearchCV


```
In [592]: # Listing parameters to test
xgb_params = {'max_depth': [None, 3, 10, 20],
              'learning_rate': [.1, .05],
              'min_child_weight': [1, 5, 10],
              'subsample': [.3, .5, .7, 1],
              'n_estimators': [100, 200, 300]}

# Creating the classifier
xgb_clf = XGBClassifier(n_jobs=-1, random_state=42)

# Feeding the parameters into the randomized search for testing
xgb_random = RandomizedSearchCV(xgb_clf, xgb_params, n_iter=10, scoring='recall_micro',
                               cv=3, random_state=42, n_jobs=-1)

# Fitting to the training data
xgb_random.fit(X_train_smote, y_train_smote)

# Print the best parameters after fitting
print(f"Best XGBoost Params (Randomized Search): {xgb_random.best_params_}")
```

Best XGBoost Params (Randomized Search): {'subsample': 0.3, 'n_estimators': 200, 'min_child_weight': 1, 'max_depth': 20, 'learning_rate': 0.05}

```
In [594]: best_parameters = {'subsample': 0.3, 'n_estimators': 200, 'min_child_weight': 1, 'max_depth': 20, 'learning_rate':
final_xgb_classifier = XGBClassifier(
    objective='multi:softmax',
    num_class=len(np.unique(y_train)),
    eval_metric='mlogloss',
    random_state=42,
    **best_parameters
)

# Train the XGBoost model
final_xgb_classifier.fit(X_train_smote, y_train_smote)

# Make predictions on the processed test data
y_pred_xgb_smote = final_xgb_classifier.predict(X_test_final)

# Calculate and print the accuracy
accuracy_xgb = accuracy_score(y_test, y_pred_xgb_smote) * 100
print(f"XGBoost Model Accuracy: {accuracy_xgb:.2f}%")

# Generate and print the classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred_xgb_smote))
```

XGBoost Model Accuracy: 79.49%

Classification Report:

	precision	recall	f1-score	support
0	0.83	0.84	0.84	9724
1	0.41	0.48	0.44	1293
2	0.83	0.78	0.81	6803
accuracy			0.79	17820
macro avg	0.69	0.70	0.70	17820
weighted avg	0.80	0.79	0.80	17820

Observation:

The overall model accuracy has improved upon tuning it with RandomizedSearchCV, as compared to the baseline XGBOOST model. the accuracy improved slightly from 79.18 to 79.49%.

the recall values of the functioning wells was 84% times making accurate predictions, and 78% times accurately predicting the non functional wells, it predicts pretty low in the functional need repair with recall score of 48% which is low.

CONCLUSION

These project successfully built a machine learning model to predict the functional status of water wells in Tanzania. By using an XGBoost classifier and addressing class imbalance with SMOTE, the model achieved a solid overall accuracy of 79%. It performs very well in identifying functional and non-functional wells. However, the model has a critical weakness: it struggles to correctly identify wells that are functional but need repair. This is evidenced by the very low recall of only 48% for this class, meaning the model misses a large majority of the wells it was designed to help maintain proactively.

RECOMENDATION

To improve the model's performance, particularly on the "needs repair" class, the following steps should be taken:

1. Refining Class Imbalance Handling: The current approach with SMOTE may be introducing noisy or unrealistic data points that confuse the model. trying another alternative strategy such as:
 - Class Weighting: Pass `scale_pos_weight` in XGBoost to directly give more importance to the minority class during training.
 - Alternative Oversampling: Explore other techniques, which can be more effective than basic SMOTE.
2. Conducting Deeper Feature Engineering and Selection:
 - by removing features with very low importance to simplify the model and potentially improve its performance.
3. Broadening Hyperparameter Search: The `n_estimators` parameter was at the edge of the search range in your randomized search. This suggests that a larger number of trees could lead to better performance. Expand your search to include a wider range of values (e.g., [200, 300, 400, 500]) to see if the model's performance continues to improve.
4. Explore Alternative Models: While XGBoost is a powerful model, others may be better suited for this specific problem. Consider training and evaluating a LightGBM Classifier, which is known for its speed and ability to handle large datasets and imbalanced classes effectively.

In []: ▶